TDT4173 · MODERN MACHINE LEARNING IN PRACTICE

Learning Hydropower Dispatch from an Optimizer

Economics-aware feature engineering and heterogeneous ensembling for Tokke–Vinje unit commitment

Kaggle team	**[25] Nah, i'd win**
Students	Francesco Binni - 169519
	Cesare Tomasi - 169206
	Cristina Zanetti - 168830
Reproducible artifact	**Short_notebook_1.ipynb, Short_notebook_2.ipynb**
Course / institution	TDT4173, Norwegian University of Science and Technology
Report date	October 2026

ABSTRACT

The task is to imitate the weekly unit-commitment decisions of SINTEF's SHOP optimizer for 14 generating units in the Tokke–Vinje hydropower system. Each daily-starting case contains a 168-hour planning horizon, yielding 2,352 binary decisions. We developed an economics-aware representation combining hourly prices, terminal marginal water values, the individual states of 17 reservoirs, hydraulic connectivity, water-availability budgets and environmental constraints. A set of gradient-boosted trees and sequence networks was evaluated using purged temporal validation. The selected prediction pipeline combines XGBoost, LightGBM, a bidirectional-GRU/attention network and a second network conditioned on out-of-fold tree predictions. A final pruning step reduced 220 candidate features to 101 without loss of accuracy, and the final pipeline achieved a **public Kaggle micro ROC-AUC of 0.99213**. Ablation and SHAP analyses show that accurately representing the entire cascade, rather than simply increasing model complexity, was the principal source of improvement. The analysis also identifies unit-switching hours and tightly coupled plants as the dominant remaining challenges.

PREDICTION TASK	BEST PUBLIC SCORE	FINAL SYSTEM
14 units × 168 hours	**0.99213** micro ROC-AUC	4 models, 101 features

Contents

1 Introduction 3

2 Problem definition and data 3

3 Exploratory data analysis 4

4 Approach and evaluation protocol 7

5 Feature engineering: from aggregates to physical state 8

6 Models and development decisions 14

7 Hyperparameter Selection 16

8 Model interpretation 17

9 Results and discussion 21

10 Reproducibility and conclusions 22

A Inventory of development experiments 22

B Complete feature dictionary (101 variables) 23

Alternative submission: Short Notebook 2 26

4 Approach and evaluation protocol 26

5 Feature engineering: a complementary perspective 27

6 Models and development decisions 31

7 Hyperparameter selection 33

8 Model interpretation 33

9 Results and discussion 35

10 Reproducibility and conclusions 36

1 Introduction

Short-term hydro scheduling determines whether using water to generate electricity immediately is preferable to retaining it for a later period. The answer depends not only on the current electricity price, but also on the marginal value of stored water, the topology of the river system, turbine characteristics, competing downstream uses and environmental requirements. Unit commitment is therefore a coupled temporal decision problem rather than 14 independent binary classification tasks.

In this project, we develop a machine learning surrogate model to approximate the unit-commitment decisions produced by SHOP., the optimizer used to construct the competition labels. The aim is *not* to recover actual historical dispatch or predict future electricity prices. For each given optimization instance, the model predicts which generators SHOP would commit during each hour of the following 168-hour planning horizon. The final artifact described here is `Short notebook 1.ipynb`, which starts with the original competition files and produces the submitted probability matrix.

The central finding is methodological: the largest gains came from representing the inputs the way the optimizer values water, rather than from more complex models. With features built on the price–water value margin, the state of every reservoir and the physical constraints of the system, an ordinary XGBoost model exceeded 0.988 public AUC, and a diversity-based ensemble reached 0.99168, then 0.99213 after reducing the features from 220 to 101. This report explains the evidence and experimental design behind that progression.

2 Problem definition and data

2.1 Target, cases and evaluation

Let c denote a scheduling case, $u \in \{1, \dots, 14\}$ a generating unit and $h \in \{0, \dots, 167\}$ an hour in that case. The binary target is

$$y_{c,u,h} = \begin{cases} 1 & \text{unit } u \text{ is committed by SHOP in hour } h, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

For example, c = Run No 1200 is the weekly case starting on 14 April 2018, $u = 2$ is Tokke_G1 and $h = 40$ is the hour from 16:00 to 17:00 UTC on 15 April; the label $y_{1200,2,40} = 1$ means that, in the plan SHOP computed for that week, Tokke_G1 is committed in that hour.

The model outputs a score $\hat{p}_{c,u,h} \in [0, 1]$. Performance is measured by *micro ROC-AUC*: all 14×168 decisions from all assessed cases are pooled before computing the ranking metric. Consequently, the entire distribution of scores matters, not merely classifications at threshold 0.5.

2.2 Inputs and physical interpretation

The provided competition dataset comprises hourly day-ahead prices; hourly reservoir and river inflows; daily starting reservoir volumes; synthetic marginal water values; hourly minimum flows and minimum volumes; and a YAML representation of reservoir capacities, turbine characteristics and hydraulic connections. The labels are provided in `Unit_commitment_decisions.csv`: for each case, they are the on/off decisions computed by SHOP, SINTEF's hydropower scheduling optimizer, for every unit and every hour of the week, given the inputs listed above. All input time series are aligned in UTC, so that hour h of a price, an inflow or a constraint refers to the same hour as the corresponding decision.

Although SHOP solves a complex optimization problem over the whole system, a large part of its decisions can be approximated by a single economic quantity, which played a key role in this project: the *generation*

Table 1. Dataset and submission structure. Consecutive cases start one day apart and their seven-day horizons overlap by 144 hours.

Property	Value	Description
Labeled cases	2,922 daily starts, 2015–2022	Training cases with known SHOP decisions, one per day from 1 January 2015 to 31 December 2022.
Unlabeled competition cases	725 daily starts, 2023–2024	Test cases to be predicted, one per day from 1 January 2023 to 25 December 2024; their decisions are not provided.
Horizon	168 hours per case	Each case is a one-week schedule (7 days $\times$ 24 hours) starting at 00:00 UTC.
Units / labels per case	14 / 2,352	Each of the 14 generating units has one binary on/off decision per hour, giving 14×168 labels per case.
Training observations after expansion	6,872,544 case–unit–hour records	Labeled data reshaped into one row per (case, unit, hour): $2{,}922 \times 2{,}352$ training examples.
Submission structure	725 rows $\times$ 2,353 columns	One row per test case: `Run No` followed by the 2,352 predicted probabilities.
Competition metric	Micro-averaged ROC-AUC	All predictions of all test cases are pooled into a single ROC-AUC.

margin, i.e. the difference between the electricity price in a given hour and the value of the water that the unit would use,

$$m_{c,u,h} = P_{c,h} - W_{r(u),\, d(c)+7}, \tag{2}$$

where P is the hourly electricity price, $r(u)$ is the reservoir feeding unit u, and $W_{r,d+7}$ is the water's terminal marginal value at the end of the seven-day horizon. The project data documentation defines the end-of-horizon value as an input to the scheduling case; the necessary $d + 7$ dates are available for the competition period. A positive margin is economically favorable to generation, but not a complete commitment rule: start costs, hydraulic couplings, storage limits, and minimum discharges can alter the decision.

The margin is not a new quantity: it expresses the standard principle of hydropower scheduling that water is released when the electricity price exceeds the value of keeping it in the reservoir. We report it here because it is the central idea of this work: it explains a large part of the observed decisions and it guides the rest of the report. It is examined empirically at the start of the feature engineering (Section 5.2), it is the basis of the main feature families in the feature engineering, together with its temporal context and normalized variants, and the model interpretation shows that the trained trees respond to it with a marked change close to zero.

The target is generated through optimization rather than by observing historical plant operation. That distinction matters for feature design: historical measurements are descriptors of the optimization instance, not consequences of SHOP's predicted unit-commitment schedule.

3 Exploratory data analysis

3.1 Completeness and consistency of the data

The input tables are complete: no file contains missing values in the core prices, inflows, water values or constraints; the hourly day-ahead price series for 2015–2024 has no gaps or duplicated timestamps, and each of the completed cases comes with a full set of ON/OFF decisions for the 14 units and the 168 hours of the week.

These decisions are not observed plant operations: they are the solutions computed by the SHOP optimization software used in the analyzed energy grid system for each weekly case, so they reflect the cost and

constraint structure of the optimization model rather than the choices of real operators. These decisions show a persistent pattern rather than an erratic one: in about 59% of the unit-weeks a unit never changes state (roughly a third always ON), and when it does run, its running periods typically last many hours (median 18 hours, with isolated one-hour runs being rare), which is consistent with energy plants that have start-up costs (Figure 1.a and Figure 1.b).

The prices look plausible for a day-ahead market: they are free of long runs of identical values, they remain positive for almost the whole period, and negative values appear only in a small share of hours in 2020–2021 and, slightly more often, in 2023–2024. The very high prices, with spikes close to 900 e/MWh, even if very rare, were kept since they seem to reflect real market conditions rather than recording errors (Figure 1.c).

The optimizer's calculation time was excluded because it is an output of the software, not a usable feature.

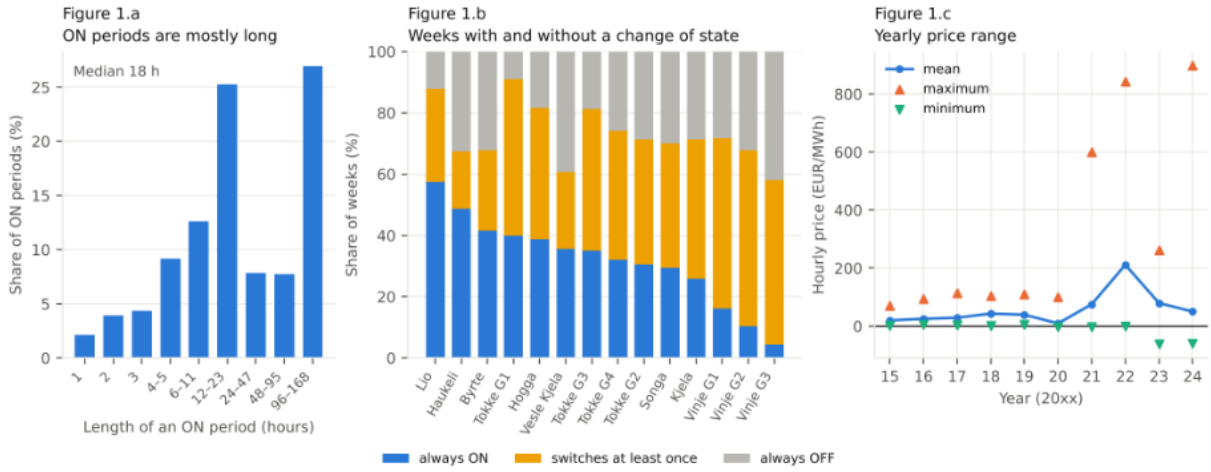


Figure 1. (a) Distribution of the length of ON periods; (b) share of weeks in which each unit is always ON, always OFF or switches at least once; (c) yearly mean, maximum and minimum of the hourly day-ahead price.

3.2 The 2022 regime

Price and water value tend to follow each other closely over the years (Figure 2.a): both stay low and stable until 2019, then decrease to a minimum in 2020 and then rise steeply in 2021 and 2022, when the average price (211 e/MWh) is far higher than in any earlier year and more than a third of the days exceed 200 e/MWh. It is worth noting that 2022 is also the year with the fewest units running. However, this is not a contradiction: the water value rose even more than the price, so keeping the water in the reservoirs became more attractive than selling it. The hydrological inputs do not clearly explain the change, because for example inflow in 2022 was similar to that of 2019, a year with moderate prices, and most large reservoirs, such as Bandak, Totak and Bordalsvatn, were filled as in other years; instead Songavatn was lower than in any other year, which can raise the local water value but does not account for the market price. The shift therefore appears to come from outside the watercourse, presumably from the electricity market itself, which is consistent with the energy crisis of that year. In 2023 and 2024 prices return to much lower levels, so the period for which predictions are required follows a different regime from the last training year.

3.3 Hydrology and seasonality

Inflows follow a clear and repeatable seasonal cycle that matches the climate of the region: they are low in winter, when precipitation falls as snow, and reach their maximum in May with the snowmelt. The reservoirs do not react at the same time, in fact their fill levels rise through the summer and peak in autumn,

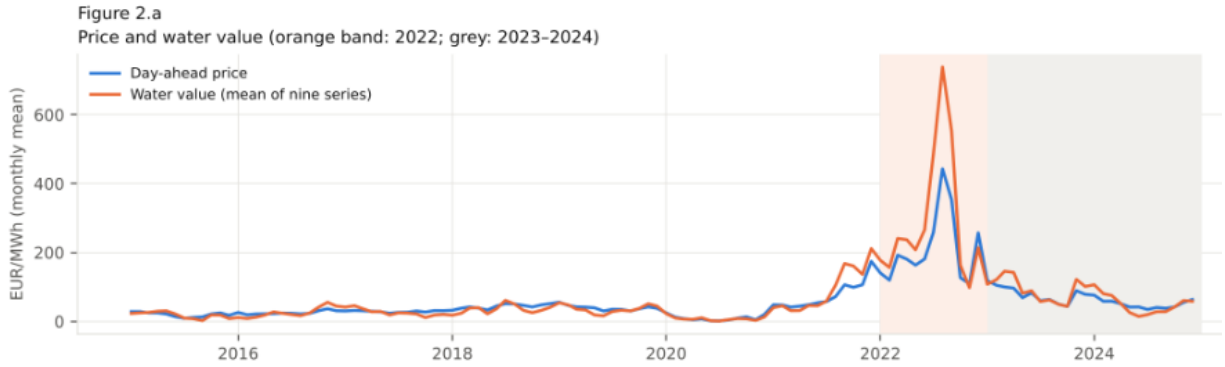


Figure 2. Monthly mean of the day-ahead price and of the water value 2015–2024. The orange band marks 2022, the grey band the test period 2023–2024.

which suggests that the snowmelt is stored and used in the following months (Figure 3.a and Figure 3.b). The fill levels of the large reservoirs are moderately correlated with each other and with the time of year, and the water value tends to be lower when the reservoir that feeds a plant is fuller, as one would expect if abundant water is worth less. This link is, however, quite loose, indeed at the same fill level the water value can differ widely between years, and the relation between price and water value is visibly weaker in 2023–2024 than before. By contrast, the inflow of a given week shows only a weak and inconsistent association with whether a unit is running, and the monthly share of ON hours has no profile that repeats from one year to the next. The seasonality of the water cycle is thus not mirrored in the unit decisions, which seem to depend more on market conditions and on the stored water than on the current inflow.

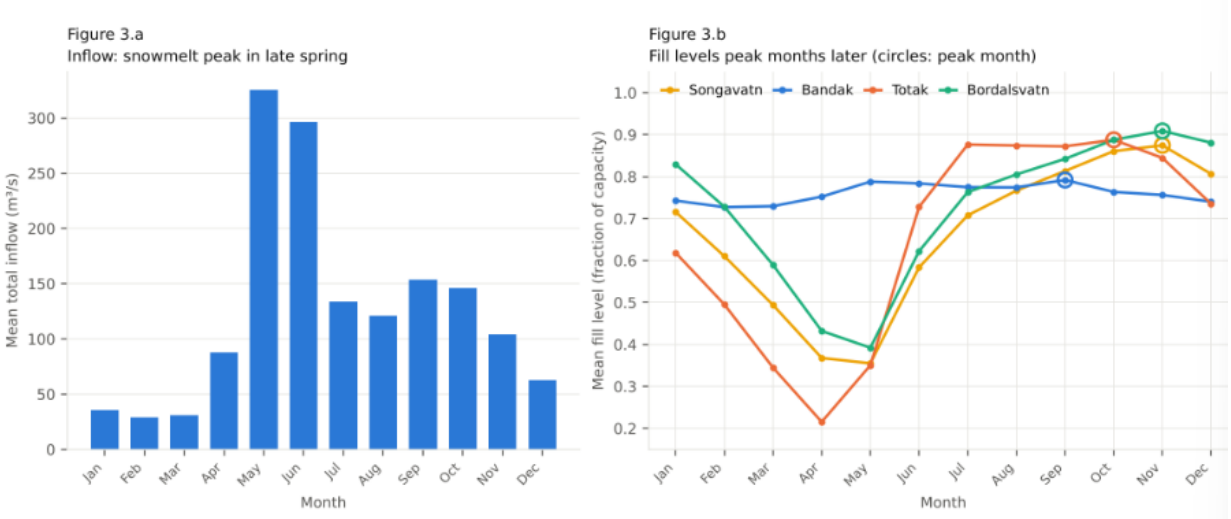


Figure 3. (a) Mean total inflow by month; (b) mean fill level of the four largest reservoirs by month, as a fraction of capacity (circles mark the peak month).

3.4 Temporal patterns

The hourly price profile has a morning and an evening peak, which became more pronounced after 2019; in 2023–2024 a deep dip appears around midday and the evening peak is the highest of the day (Figure 4.a). The units follow this rhythm, with a deeper midday pause and a stronger evening peak in 2020–2022 than in 2015–2019, although the differences are small (Figure 4.b). Units are also ON less often at weekends, and this pattern is found in every year (Figure 4.c). Apart from the weekend effect, calendar variables show only modest differences in the share of ON hours, and the month-to-month pattern changes too much between years to be considered a seasonal rule. Finally, consecutive cases cover almost the same

week (they share six of seven days) and their decisions agree for most of the shared hours, so neighboring cases are not completely independent observations.

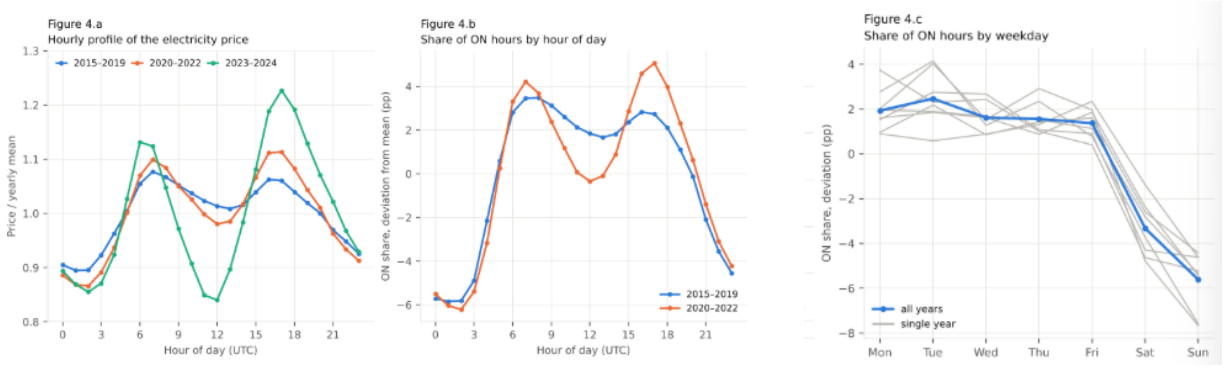


Figure 4. (a) Hourly price profile relative to the yearly mean; (b) share of ON hours by hour of the day, as deviation from the mean; (c) share of ON hours by weekday, as deviation from the mean, for all years together and for each single year.

4 Approach and evaluation protocol

4.1 Overview of the approach

The goal is to predict, for every case, unit and hour, the probability that SHOP commits the unit. Our approach consists of four steps: (i) represent the data so that each decision to predict is a separate example; (ii) build features that describe the economic and hydrological situation of each unit in each hour; (iii) train two different families of models on these features and combine their predictions; (iv) evaluate every choice with a temporal validation that imitates the real task of predicting future years.

Data representation. In the original decision file each case is a single row containing all its 2,352 decisions. We transform the data into an intermediate *long* format with one row per decision, i.e. per combination of case, unit and hour, containing the features of that unit in that hour and, for the training cases, its 0/1 label. This format is required mainly by the tree models described below, which learn a mapping from a fixed list of feature values to a single target: each training example must therefore be one row with its own label. It also lets every row carry the information that is specific to its unit and hour (for instance the margin of that unit's upstream reservoir), and turns the 2,922 labeled cases into about 6.9 million training examples. The construction of this table is detailed in the feature engineering section.

Models. The final prediction combines four models:

- **XGBoost** and **LightGBM**: two implementations of *gradient-boosted decision trees*, i.e. ensembles of decision trees built one after another, each tree correcting the errors of the previous ones. They predict each row of the long table independently.
- a **sequence neural network**: instead of one row at a time, it receives a whole case as a grid of 14 units $\times$ 168 hours, so that it can use information from the other hours of the week and from the other units when predicting each decision.
- a **refiner** network: the same sequence network with one additional input, the probability predicted by XGBoost, which it learns to correct using the context of the whole week.

The final prediction is a weighted average of the four models. Their architectures, hyperparameters and the other models tried during the project are described in the following sections.

4.2 Validation strategy

To choose features, models and hyperparameters, each candidate must be evaluated on data that it has not seen during training. A common approach is *cross-validation*: the labeled data are split into parts (*folds*), the model is trained on some folds and evaluated on the remaining one, and the procedure is repeated so that each fold is used once for evaluation.

In this dataset the folds cannot be chosen at random. A new case starts every day and covers seven days, so two consecutive cases share 144 of their 168 hours: the same prices, inflows and constraints, and largely the same SHOP decisions. If cases were assigned to folds at random, almost every validation case would have its immediate neighbours in the training data, and the model would be evaluated on situations it has essentially already seen. The validation score would then be too optimistic and would favor models that memorize neighbouring weeks rather than models that generalize. A random split would also train on cases that come *after* the validation cases, whereas the competition asks to predict 2023–2024 from earlier years, and the conditions change strongly over time (e.g. the price regimes discussed above).

Our main model-selection procedure therefore uses *expanding-window temporal cross-validation*, which reproduces the real situation of predicting a future period from the past. One year at a time is used for validation: 2020, 2021 and 2022. To validate year Y, the model is trained only on earlier cases, so the training period grows ("expands") from one fold to the next. In addition, a seven-day *purge* removes the cases near the boundary: a case starting on 28 December of the previous year would cover the first days of Y, so only cases whose seven-day windows end no later than the start of Y are kept for training. This prevents any overlap between training and validation horizons.

The score reported for a configuration is the mean of the three yearly micro ROC-AUC values. The final models are then trained on all labeled 2015–2022 cases, with the hyperparameters and the numbers of epochs or boosting rounds selected during this validation. Training labels from 2023–2024 are never available.

Because many configurations were compared on the same three validation years, there is a risk that the selected one is the best on those years partly by chance (*overfitting the validation*). To check this, we performed a *retrospective check on 2019*, a year not used in the final phase to choose features, hyperparameters or blend weights. The final prediction pipeline was trained with exactly the same settings as for the submission, but only on the cases up to 2018 (with the same seven-day purge), and then evaluated on 2019. If the choices made on 2020–2022 still hold on 2019, they are unlikely to depend on those specific years; the results are reported in the results section. However, 2019 is not a completely untouched year, since it was examined during the exploratory analysis and early experiments, so this check is best viewed as a corroboration rather than a fully independent test set.

5 Feature engineering: from aggregates to physical state

5.1 Data Representation

Before training the models, the data had to be reorganized into a format suitable for machine learning. The main challenge was that the different datasets did not share the same structure. The unit commitment decisions were stored with one row per weekly case, containing 2,352 binary values representing the ON/OFF state of each of the 14 generators over 168 hours. The input data, however, were organized separately as time series, with hourly measurements for electricity prices, water inflows and environmental constraints, and daily measurements for reservoir volumes and water values.

To combine these sources, we first associated each weekly case with the corresponding input data. For a case beginning on day d, we collected the electricity prices, inflows and constraints covering the following 168 hours, together with the reservoir volumes and water values at the beginning of the week. We also

included the water values at day $d + 7$, which represent the terminal value of the stored water at the end of the scheduling horizon.

Once the information was aligned, we reorganized it so that **each row represented a single prediction: whether a particular generator would be ON or OFF at a particular hour of a particular weekly case**. This representation, commonly referred to as *long format*, makes it possible to associate each decision with the information needed to predict it.

Not all the input information changes for every generator or every hour. For example, the initial volume of a reservoir remains the same throughout a weekly case, while the electricity price changes every hour but is shared by all generators. When constructing the dataset, these values were therefore repeated wherever relevant. Other quantities, such as the difference between electricity price and the water value associated with a generator, were calculated separately for each generator and hour.

Since each weekly case contains 14 generators and 168 hours, it produces 2,352 rows. Across the 2,922 training cases and 725 test cases, the resulting dataset contains:

$$3{,}647\,\text{cases} \times 14\,\text{generators} \times 168\,\text{hours} = 8{,}577{,}744\,\text{rows}. \tag{3}$$

Each row contains 101 features describing the economic conditions, reservoir states, hydraulic constraints and other factors relevant to the scheduling decision. Of these rows, 6,872,544 belong to the training data, while the remaining 1,705,200 correspond to the test cases.

This representation is used directly by XGBoost and LightGBM, which learn to predict individual generator-hour decisions. The neural networks use the same information but organize it differently: instead of treating each generator-hour as a separate example, they process an entire weekly case at once, allowing them to consider the relationships between generators and how operating conditions evolve throughout the week.

Importantly, all features are constructed exclusively from the provided input data and the hydropower system topology, without using the target decisions as inputs.

5.2 The generation margin as a first-order explanation

We divided the labeled hourly observations into groups based on the generation margin defined in Eq. (2). For each group, we calculated the percentage of times the generator was ON. For several plants, the switch from low to high ON probability is concentrated near a zero margin (Fig. 5). As a standalone *ranking score*, without machine learning, the margin gives within-unit AUCs of approximately 0.989 for Byrte, 0.986 for Vesle Kjela, and 0.972 for Songa. It is much less adequate for cascade-sensitive units (approximately 0.76–0.78 for the three Vinje units). The Tokke–Vinje system is a cascade: water released by one plant flows into a downstream reservoir and is used again by the next plant (e.g. Vinje discharges into Vinjevatn, which feeds Tokke), so the decision for a unit in the middle of the cascade also depends on the state of reservoirs and plants elsewhere in the system, not only on its own intake. This contrast motivated a model that represents both local economics and effects elsewhere in the river system.

5.3 Feature Engineering: Economic and Hydraulic Information

Feature engineering played a central role in improving the predictive performance of our models. Rather than simply generating a large number of additional variables, our objective was to identify which information is actually relevant to the decisions made by SHOP and represent it in a form that machine learning models can effectively exploit. Our experiments showed that two aspects were particularly important: the economic conditions that determine whether producing electricity is profitable, and the hydraulic structure of the system, which determines how water can be stored, transferred and used by different power plants.

The first fundamental aspect is the economic relationship between electricity prices and water values. As discussed in Section 2.2, hydropower scheduling involves deciding whether to use the available water

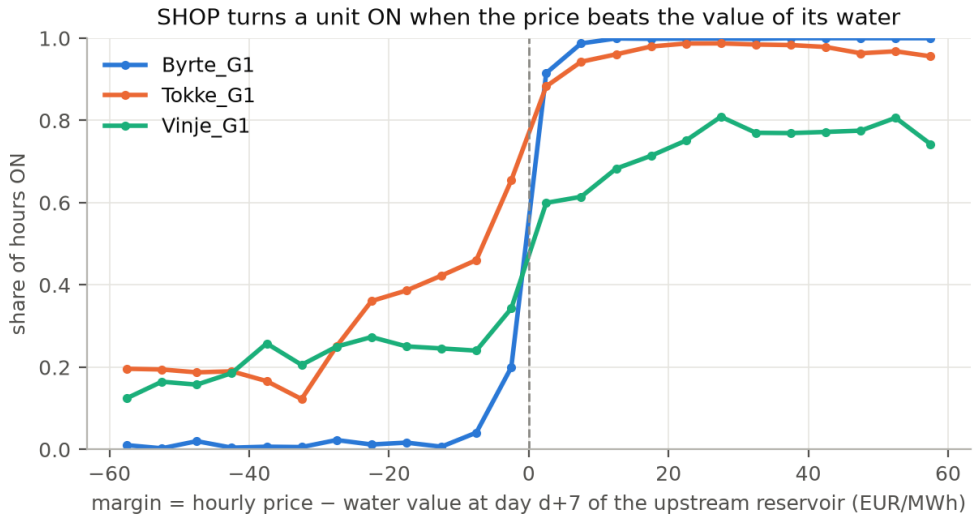


Figure 5. Fraction of labeled hours committed as a function of hourly price minus terminal water value; bins of 5 e/MWh. The nearly threshold-like behavior for Byrte contrasts with the more coupled behavior of Vinje.

to generate electricity immediately or preserve it for future production. The electricity price represents the revenue that can be obtained by producing energy at a given hour, while the water value represents the economic value associated with retaining that water. The difference between these two quantities, expressed by the economic margin in Eq. (2), therefore provides an important indication of whether operating a generating unit is economically attractive.

When the electricity price exceeds the relevant water value, generating electricity becomes potentially profitable. Conversely, when the price is lower, preserving the water may be preferable. However, this relationship is not sufficient to determine the final ON/OFF decision. Starting and stopping a generator involves additional costs, and a unit may remain OFF even during an hour with a positive margin if the expected profit is too small to justify starting it. Similarly, a generator may continue operating during a temporarily unprofitable hour if shutting it down and restarting it shortly afterwards would be more expensive.

For this reason, the model needs to understand not only the economic margin at the current hour, but also how it evolves throughout the scheduling horizon. We therefore introduced several features derived from Eq. (2), including moving averages, minimum and maximum values over surrounding hours, the proportion of profitable hours within a day or week, and the distance to changes in the sign of the margin. These features help distinguish an isolated profitable hour from a longer period of favorable market conditions. This distinction is particularly important because the optimizer considers the profitability of an entire operating schedule rather than making independent decisions for each hour.

The second fundamental aspect is the hydraulic topology of the system. Unlike independent generating units, the power plants in Tokke–Vinje are connected through a network of reservoirs, tunnels and waterways. Water released by an upstream plant may become available to a downstream plant, meaning that the operating decision of one generator can influence the possibilities and economic value of production elsewhere in the system. Consequently, predicting whether a unit will operate requires information not only about the reservoir from which it directly receives water, but potentially also about other reservoirs connected through the hydraulic network.

A natural way to include topology is to describe, for each unit, the reservoirs directly feeding it, those further upstream and those downstream, through aggregated quantities such as total stored volume, total inflow and weighted average water value. Such features are useful, but aggregation introduces an important limitation: information about individual reservoirs is partially lost before reaching the model.

For example, two different configurations of reservoir volumes could produce the same aggregated total,

even though they might lead to different operating decisions. Moreover, the importance of a reservoir is not necessarily determined by its direct connection to a particular plant. Vinjevatn illustrates this problem particularly well: it receives water released by Vinje and Haukeli while also supplying Tokke. Its state can therefore influence the decisions of several plants in different parts of the cascade, even when they do not draw water directly from it.

To overcome these limitations, **we provided the model with the individual state of all 17 reservoirs**. For each reservoir, we included its initial volume, its water value at the beginning and end of the scheduling horizon, the change between these values, its average weekly inflow and the difference between the hourly electricity price and its terminal water value. This produced 102 features that describe the reservoirs separately, rather than reducing their information to a few predefined aggregates. Many of them later turned out to be duplicates or almost unused (for example, the 17 reservoirs have only nine distinct water-value time series, meaning that several reservoirs share identical water-value features), and the pruning step described at the end of this section reduced them to 19.

The advantage of this representation is that the model can learn which reservoirs are relevant for each generator and under which conditions. For example, a decision tree can learn that the water value or filling level of Vinjevatn is particularly important for certain units, while different reservoirs may be more informative for others. This flexibility would be much more limited if the model only received a weighted average combining several reservoir states. The importance of this design choice was confirmed experimentally: removing the individual reservoir-state features from the full 220-feature version reduced the mean ROC-AUC in temporal cross-validation by approximately 0.0114, one of the most significant effects observed in our feature ablation experiments.

Providing individual reservoir information does not mean abandoning the aggregated topological features. On the contrary, **the final representation combines detailed reservoir states with additional information derived from the hydraulic connections between them**. Some reservoirs are connected by tunnels and effectively behave as a single storage system. In these situations, considering each reservoir separately is not sufficient to describe the total amount of water available to a plant. We therefore introduced the concept of *effective storage*, combining hydraulically connected reservoirs to estimate their collective water availability, filling level and risk of overflowing.

The risk of overflowing, or *spill*, is particularly relevant because water that cannot be retained may otherwise be lost without producing electricity. A plant may therefore have an incentive to operate even when electricity prices are not especially favorable, simply to create additional storage capacity or avoid wasting incoming water. By explicitly representing these conditions, the model can learn operating decisions that cannot be explained by the economic margin alone.

Another important improvement concerns how water availability is expressed. Reservoir volumes are measured in millions of cubic meters, but the same volume can have very different implications depending on the size of the power plant. A relatively small reservoir may contain enough water to operate a small generator for several days, while the same amount might support a much larger plant for only a few hours.

To make this information more meaningful, we introduced a set of **hydraulic budget features** that express water availability in terms of equivalent hours of turbine operation. These quantities are calculated by relating stored water volumes and expected inflows to the maximum discharge capacity of the corresponding turbines. For example, a hydraulic budget of 12 hours indicates that the available amount of water is equivalent to approximately 12 hours of operation at maximum turbine discharge. This representation makes water availability easier to interpret and allows the model to compare operating conditions across plants of different sizes. Similar calculations are used to describe the remaining storage capacity and the amount of water potentially exceeding reservoir limits.

Finally, we improved the representation of **environmental and operational constraints**. Hydropower plants cannot decide when to release water based exclusively on economic profitability. They must also respect restrictions such as minimum river flows and minimum reservoir volumes. These requirements may

vary during the week and can directly influence whether a generator must operate, even when electricity prices would otherwise suggest keeping it OFF.

In the first versions of our feature set (Section 5.4), these constraints were summarized using weekly averages. However, an average cannot tell the model precisely when a restriction becomes relevant. In the final version, minimum-flow requirements are therefore represented hour by hour, together with additional features describing the margins relative to minimum-volume constraints. This allows the model to associate a specific operating decision with the environmental requirements applying at that particular time. These features proved especially useful for Lio, whose operation is closely related to the minimum-flow requirements of the river into which it releases water.

Overall, these ideas produced **220 candidate features**. After removing duplicates and features that the models almost never used (see the end of this section), the final feature set contains **101 features**, summarized in the table below, combining economic indicators, individual reservoir states, hydraulic topology, effective storage, water availability and environmental constraints. These groups provide complementary information: economic features help identify when production is financially attractive, while hydraulic and constraint-related features describe whether production is physically possible, necessary or advantageous within the interconnected system.

The main lesson from this process is that improving the representation of the problem was more effective than simply increasing model complexity. By providing the models with information that more closely reflects the economic and physical mechanisms underlying SHOP's decisions, we enabled them to identify relationships that would remain hidden in a more aggregated feature representation.

Table 2. Final feature inventory, corresponding to the extraction code in `Short notebook 1.ipynb`. The middle column gives the number of features in the final set and, in brackets, among the 220 candidates before pruning. The complete list of the 101 features is in the appendix.

Family	n (cand.)	Role and representative variables
Individual reservoir state	19 (102)	$P - W_7$ for the reservoirs with their own water-value series; W_0, W_7, ΔW of key reservoirs such as Vinjevatn and Byrtevatn; V_0 of Byrtevatn; weekly inflow of Vaamarvatn
Weekly-mean constraints	3 (9)	Mean minimum flows of Byrtevatn and of the river Vest Vassdraget; mean minimum volume of Byrtevatn
Hourly price and position	13 (22)	Price, rank, z-score, moving context, weekly spread, hour and unit index
Unit-level economics	12 (16)	Upstream/downstream valuations and net margins
Margin and context	22 (24)	m, moving averages/extrema, positive-margin fractions, distances to sign changes
Effective storage and unit data	13 (20)	Fill, spill risk, hourly inflow, turbine size and start cost
Hydraulic budget	8 (9)	Available inflow, storage and spill/headroom in hours of turbine operation
Hourly environmental limits	8 (15)	Hourly minimum flows of Byrtevatn and Vest Vassdraget, the Lio receiving-river deficit, minimum-volume margins
Normalized margin	3 (3)	Margins scaled by price volatility or water value
Total	**101 (220)**	

5.4 Incremental experiments and ablation

The feature system evolved in three successive versions. Version 1 retained the individual reservoir information and economic-margin context. Version 2 added effective-storage physics and unit properties. Version 3 added hydro budgets, hourly environmental constraints and normalized margins. With a com-

mon depth-8 XGBoost evaluation, mean 2020–2022 AUC increased from 0.97598 to 0.97761 to 0.97994. A subsequent depth-10 configuration yielded approximately 0.98030.

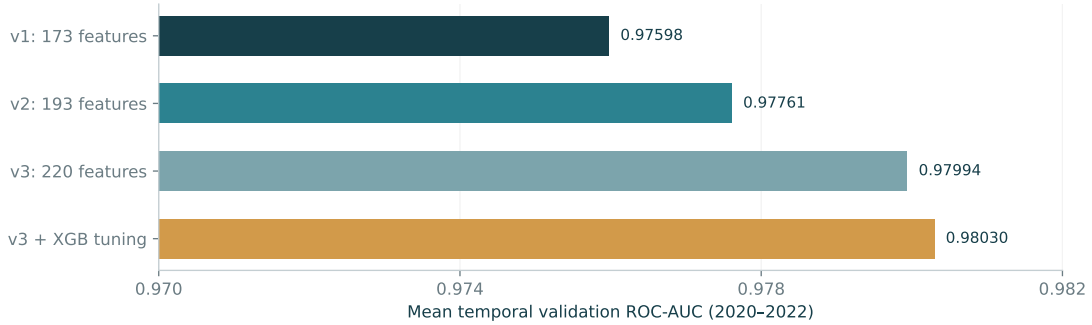


Figure 6. Temporal validation improvement from successive feature stages. The final bar also reflects XGBoost hyperparameter adjustments, so it is not a pure feature-only comparison.

The central ablation is particularly informative. Removing the *raw individual reservoir states* from the 220-feature v3 model reduced mean AUC from 0.97994 to 0.96853 (-0.01141). Removing only the explicit temporal margin-context group reduced it to 0.97954 (-0.00040); removing both gave 0.96810. This does not mean temporal context is irrelevant: its content is partially recoverable from the other highly correlated margin features. It does show that preserving the identity of each reservoir is materially more valuable than further elaborating an already informative margin signal.

Other candidates were investigated but not retained: historical inflow anomalies, additional price statistics, cyclic calendar encodings, estimated heads, alternate upstream/downstream combinations, a cascade-adjusted energy threshold, and a price-rank-versus-inflow-budget rule. They did not show consistent validation gains or produced weak univariate discrimination. These negative experiments helped narrow the final representation rather than expanding the feature set indiscriminately.

5.5 Pruning the feature set

Version 3 contained 220 features, and many of them were added in groups (for example, the same six quantities for every reservoir) without checking whether each one was actually useful. Features that carry no new information make the models slower, harder to interpret and more exposed to noise, so as a last step we removed the superfluous ones and measured how the ROC-AUC changed. Since the labels of the Kaggle test set are not available, every decision was based on expanding-window cross-validation, where the model was trained on an increasing number of past years and evaluated on the following year. This approach ensured that the model was always tested on future data that had not been used during training. The results were then further checked using the 2019 retrospective evaluation.

A small change in AUC is meaningful only if it is larger than the variation that occurs anyway when the same model is retrained with a different random seed. We therefore first trained the same XGBoost model[1] on all 220 features with three different seeds: the mean AUC was 0.97975, with a standard deviation $\sigma = 0.00032$ across seeds. We decided in advance to accept a reduced set only if its mean AUC over three seeds was at least $0.97975 - \sigma$ and no single validation year lost more than 2σ. Features were removed in two groups:

- **Redundant features (52).** Two features are constant, and the others are exact copies, or copies multiplied by a constant, of another feature (correlation above 0.9999). Most come from reservoirs that share the same water-value series, or whose inflows are proportional to each other. A decision tree splits on thresholds, so such a copy cannot give it any new information. Removing them left the AUC unchanged (0.97975).

[1]The faster configuration used for all feature comparisons in this section: depth 8, learning rate 0.05, early stopping on the validation year.

- **Rarely used features (67).** We ranked the remaining 168 features by their mean absolute SHAP value, a measure of how much each feature changes the predictions (it is explained in the model interpretation section), and removed the least important ones: together they account for only 6.9% of the total. Removing them did not reduce the overall AUC, but the AUC of Lio dropped from 0.958 to 0.954. The reason is that the features of the reservoir (Byrtevatn) and of the river (Vest Vassdraget) on which Lio depends have a small importance in total, because they matter for only one of the 14 units, but they are essential for that unit. We therefore kept these 13 features, after which the AUC of Lio returned to its original level.

Table 3. ROC-AUC with the 220 candidate features and with the final 101 features. CV is the mean over the validation years 2020–2022; 2019 is the retrospective check (training up to 2018). The blend is 0.2 XGBoost + 0.4 LightGBM + 0.4 sequence network.

Model	CV, 220	CV, 101	2019, 220	2019, 101
XGBoost, comparison configuration (3 seeds)	0.97975	0.98010	—	—
XGBoost, final configuration	0.98030	0.98050	0.98558	0.98590
LightGBM	0.98020	0.98085	0.98560	0.98549
Sequence network	0.97670	0.97925	0.98400	0.98637
Blend	0.98291	0.98382	0.98656	0.98744

The table above compares the performance of each model using all 220 candidate features and the final set of 101 selected features. None of the models showed a decrease in ROC-AUC, either during expanding-window cross-validation or in the 2019 retrospective evaluation. The small improvements should not be read as real gains, since they are of the same order as the seed-to-seed variation: the reliable conclusion is that the 119 removed features were not needed. Most public scores reported in the results section were obtained with the 220-feature version, before this step. The complete four-model pipeline was then validated again with the 101 features: it obtained a mean ROC-AUC of 0.98386 on 2020–2022 (0.98316 with 220 features), 0.98789 on the nested 2019 check (0.98685) and a public score of 0.99213 (0.99168). The differences are small, partly within the seed-to-seed variation, but they all point in the same direction, which confirms that the pruning did not cost accuracy.

6 Models and development decisions

6.1 Why multiple model classes?

Gradient-boosted trees and neural sequence networks were selected because they offer two complementary ways of approaching the unit commitment problem.

Gradient-boosted trees, such as XGBoost and LightGBM, are particularly effective with structured tabular data, making them well suited to our representation, where each row describes a specific generator at a specific hour. However, each prediction is based exclusively on the features available in that row. Although these include information about surrounding hours, such as rolling averages of the economic margin, the model can only exploit temporal relationships that have already been explicitly represented through feature engineering.

Neural sequence networks, on the other hand, can process the entire scheduling horizon, considering all 168 hours and 14 generators together. This allows them to learn temporal patterns and interactions between generating units directly from the sequence, without relying exclusively on predefined summaries. Such relationships are particularly relevant in hydropower scheduling, where operating decisions depend on preceding and upcoming conditions, and water released by one plant can influence the operation of others.

By exploring both approaches, we aimed not only to identify the strongest individual predictor, but also to determine whether their different ways of processing information could provide complementary predictions. We additionally evaluated a simple economic rule and a second-stage meta-model. Unsuccessful alternatives are summarized in Appendix A.

6.2 Tree predictors

XGBoost is the principal supervised model. It uses histogram-based binary-logistic boosting and predicts each long-format row. Three models with different random seeds are averaged. The submitted configuration uses depth 10, learning rate 0.03, 800 boosting rounds, and regularized row and column sampling. **LightGBM** is trained on the same feature table, with up to 255 leaves and 500 rounds. Its standalone validation is close to XGBoost, but the correlation between their validation predictions is high (approximately 0.995). LightGBM is therefore a modest source of variance reduction rather than an independently transformative model.

6.3 Sequence network

The neural architecture receives one complete case, represented as a 14×168 grid. The 101 tabular features are standardized from training statistics and supplemented by missingness flags (103 channels in the reported implementation). After a learned input projection and unit embedding, three blocks alternate *bidirectional GRU* processing across time with *four-head attention* across units at the same hour; feed-forward residual connections and layer normalization complete each block. A shared output head predicts one logit per unit-hour.

An initially smaller network was enlarged from hidden width 96 and two blocks to width 128 and three blocks. Although its standalone mean AUC (approximately 0.9793) remains lower than XGBoost's (approximately 0.9805), its errors are less correlated with the tree predictor (about 0.932 for XGBoost versus the sequence network in 2022). Mixing the models yields a larger improvement than further tuning either one in isolation.

6.4 Out-of-fold Refiner

Although the sequence neural network can capture temporal patterns and interactions between generators, XGBoost remains a stronger individual predictor. We therefore introduced a second neural network, called the *refiner*, to investigate whether XGBoost's predictions could be further improved by considering the broader scheduling context.

The refiner uses the same architecture as the sequence network described previously, but receives one additional input: the probability predicted by XGBoost for each generator and hour. The objective is to allow the network to learn when XGBoost's predictions are reliable and when they should be adjusted, based on the surrounding hours and the operating conditions of other generators. For example, an isolated ON prediction during a profitable hour might be corrected if the surrounding conditions suggest that starting the generator would not be worthwhile.

However, using predictions from an XGBoost model trained on the same examples would introduce a risk of *target leakage*, since those predictions would be unrealistically accurate compared with predictions on unseen data. To prevent this, we generated *out-of-fold predictions* using a leave-one-year-out strategy. For each training year, XGBoost was trained on the remaining years, excluding overlapping weekly windows around the year boundaries, and then used to predict the held-out year. This ensures that the refiner learns from predictions generated without using the corresponding target labels.

For the competition test cases, the additional input comes from the final XGBoost model trained on all labeled data. These probabilities are converted into logits and standardized before being passed to the

network. Since the leave-one-year-out procedure may use later training years, the separate retrospective evaluation on 2019 employed a nested procedure using only data up to 2018.

In this way, the refiner combines XGBoost's predictive strength with the sequence network's ability to analyze the complete weekly schedule, potentially correcting errors that are difficult to identify from individual generator-hour features alone.

6.5 Final ensemble

The final score is a fixed convex combination:

$$\boxed{\hat{p} = 0.20\,\hat{p}_{\mathrm{XGB}} + 0.30\,\hat{p}_{\mathrm{LGB}} + 0.25\,\hat{p}_{\mathrm{NN}} + 0.25\,\hat{p}_{\mathrm{refiner}}.} \tag{4}$$

Weights were compared using temporal out-of-fold validation, then checked retrospectively on 2019. The strongest submitted pipeline is this four-model blend, not the single best standalone learner.

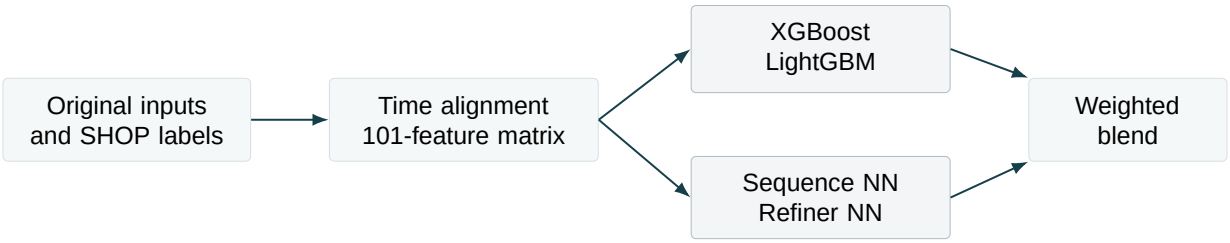


Figure 7. Reproducible final pipeline implemented in `Short notebook 1.ipynb`. During refiner training, its XGBoost input is produced out of fold; during competition inference it uses the final XGBoost probabilities.

7 Hyperparameter Selection

Hyperparameters were selected through a limited set of experiments using temporal cross-validation on 2020, 2021 and 2022. For **XGBoost**, we performed a small grid search comparing different tree depths, learning rates, minimum child weights and regularization settings. The best results were obtained with a maximum depth of 10, a learning rate of 0.03 and a minimum child weight of 50. Early stopping during validation was used to estimate an appropriate training length, leading to 800 boosting rounds in the final configuration.

For **LightGBM**, we selected parameters providing comparable model capacity and regularization, without conducting an extensive separate search. For the **sequence neural network**, we compared a small number of architectural configurations, adjusting the hidden dimension, number of blocks, regularization and training epochs. The final configuration uses a hidden dimension of 128, three blocks and 15 epochs, while the refiner adopts the same settings.

We deliberately avoided extensive hyperparameter optimization because the improvements observed after the initial experiments were relatively small and sometimes inconsistent across validation years. Further tuning would have increased computational costs and the risk of overfitting to the validation periods, without clear evidence of meaningful gains. Instead, we prioritized improving the feature representation and combining complementary models.

All selected hyperparameters are **fixed in the final short notebook**, which does not perform any additional search or tuning. This reduces execution time and ensures that the submitted pipeline follows the configurations evaluated during development.

Table 4. Settings used by the first reproducible notebook and the evidence behind them.

Component	Fixed settings	Selection rationale
XGBoost	`depth=10`; `lr=0.03`; `min_child_weight=50`; subsample 0.8; columns 0.6; $\lambda = 5$; 800 rounds; 3 seeds; 128 bins	Small temporal CV grid: depth 10 outperformed depth 6 and the initial depth 8 for the selected comparison. Fold early-stopping counts at lr $= 0.03$ were 615, 373 and 750; 800 fixed rounds were used for full data. 128 bins reduce GPU memory use.
LightGBM	255 leaves; `lr=0.05`; minimum leaf size 200; fraction of features 0.6; bagging 0.8; $\lambda_2 = 10$; 500 rounds	Similar complexity and regularization to the tree baseline; fold early-stopping counts 454, 317 and 459. Not subjected to a large independent search given its high correlation with XGBoost.
Sequence NN	width 128; 3 GRU/attention blocks; 4 attention heads; dropout 0.25; weight decay 10^{-3}; OneCycle maximum LR 10^{-3}; 15 epochs; batch 16	Earlier width 96/two-block version reached peak validation AUC after about 12–18 epochs; 15 epochs and stronger regularization were adopted. Larger architecture improved blend performance in CV.
Refiner NN	Same network settings, plus one XGBoost-logit input	Only the added input channel differs from the sequence network. LOYO with a seven-day purge protects the training channel from in-fold fitted predictions.
Blend	Weights $0.20/0.30/0.25/0.25$	Candidate weights were compared on 2020–2022 out-of-fold predictions and retrospectively checked on 2019.

8 Model interpretation

To better understand how XGBoost makes its predictions, we analyzed the importance and contribution of the 101 features used by the model. We trained the final XGBoost configuration on data from 2015 to 2021 and evaluated it on 2022, achieving a validation ROC-AUC of 0.9863.

We then applied two complementary interpretation techniques: Gain Importance and SHAP. Gain Importance measures how much each feature contributes to improving the decision trees during training, allowing us to identify which features the model relies on most. SHAP, instead, explains how individual features influence the model's predictions, indicating whether their contributions push a prediction toward ON or OFF.

For the SHAP analysis, we selected a random sample of 120,000 rows from the 2022 validation set. Since each row represents a specific generator at a specific hour, this sample covers a large number of operating conditions while keeping the computational cost of calculating SHAP values manageable. The following sections present and discuss the results of both analyses separately.

8.1 Gain Importance

The Gain Importance analysis revealed that the most influential features in XGBoost are closely related to the economic margin introduced in Eq. (2). The highest-ranked feature was `m_cmean7`, representing the average economic margin over seven consecutive hours centered on the current hour, accounting for approximately **13.4% of the total gain**. The second was `m_over_W7`, which expresses the economic margin relative to the terminal water value, contributing approximately **9.5%**. The results are illustrated in Figure 8.

The importance of these features is consistent with the economic principles underlying hydropower scheduling. In particular, the prominence of `m_cmean7` suggests that XGBoost considers not only whether generating electricity is profitable at a specific hour, but also whether favorable conditions persist over the surrounding hours. Since starting a generator involves additional costs, operating continuously during several profitable hours may be more advantageous than starting it for a single isolated profitable hour.

Similarly, the importance of `m_over_W7` indicates that the model benefits from evaluating the economic margin relative to the value of the stored water, rather than relying exclusively on its absolute magnitude. This is particularly useful when electricity prices and water values vary substantially across different market conditions.

Overall, these results suggest that XGBoost has learned relationships consistent with the economic reasoning behind SHOP's scheduling decisions, highlighting the value of the engineered features derived from the economic margin.

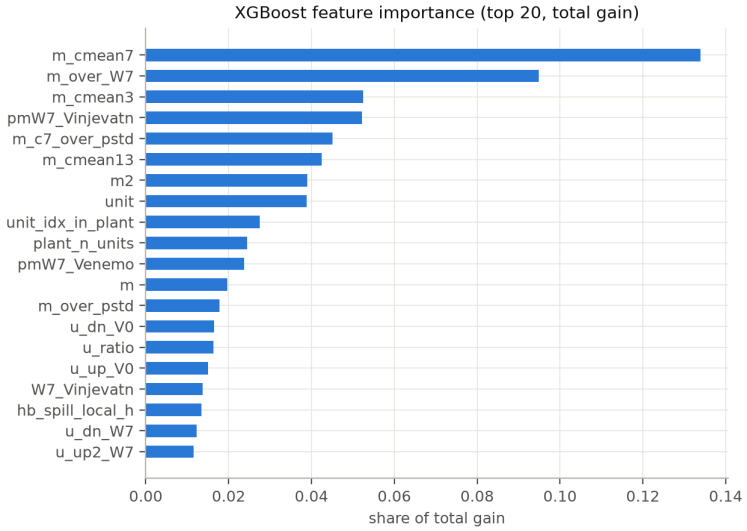


Figure 8. Top features by total XGBoost split gain (model trained on 2015–2021, evaluated on 2022).

8.2 Ranking of the most important features

The table below lists the ten features with the largest mean absolute SHAP value, together with their share of the total gain. The two measures answer slightly different questions: gain counts how much a feature was used to improve the trees during training, while SHAP measures how much it moves the predictions on the validation year. When several features carry similar information, such as the different averages of the margin, the trees can use any of them, so the importance is shared among them and the two rankings do not need to coincide.

Table 5. The ten most important features by mean absolute SHAP value (XGBoost trained on 2015–2021, 120,000 rows of the 2022 validation year), as a share of the total over all 101 features. The gain column gives the share of the total gain of the same model; features outside the top 15 by gain are marked < 1.6.

#	Feature	Meaning	SHAP (%)	Gain (%)
1	`pmW7_Vinjevatn`	Hourly price minus the d+7 water value of Vinjevatn	8.6	5.2
2	`unit_idx_in_plant`	Number of the unit within its plant (G1, G2, …)	5.5	2.8
3	`m_over_W7`	Margin m divided by the d+7 water value of the upstream reservoir	5.0	9.5
4	`m2`	Hourly price minus the d+7 water value two levels upstream	4.5	3.9
5	`unit`	Identifier of the unit	3.5	3.9
6	`pmW7_Venemo`	Hourly price minus the d+7 water value of Venemo	3.3	2.4
7	`W7_Vinjevatn`	d+7 water value of Vinjevatn	2.9	< 1.6
8	`m_cmean13`	Average of m over the 13 hours centered on the current hour	2.8	4.2
9	`p_minus_mean`	Hourly price minus the weekly mean price	2.7	< 1.6
10	`m_cmean7`	Average of m over the 7 hours centered on the current hour	2.5	13.4

The ranking is dominated by three kinds of information.

- **The economic margin and its variants** (`m_over_W7`, `m2`, `m_cmean13`, `m_cmean7`). They confirm the principle described in the data section: a unit tends to be ON when the price exceeds the value of the water it would use. The smoothed versions dominate the gain because the optimizer plans runs of several hours, so the margin around the current hour is more informative than the margin of the hour alone.
- **The water value of the central reservoirs of the cascade** (`pmW7_Vinjevatn`, `W7_Vinjevatn`, `pmW7_Venemo`). Vinjevatn receives the water released by Vinje and Haukeli and feeds Tokke, so its value affects units in different parts of the system; this is discussed further below.
- **The identity of the unit** (`unit_idx_in_plant`, `unit`). Units of the same plant draw water from the same reservoir and therefore have the same margin, but they are not operated in the same way: in the training data, for example, Vinje G1 is ON in 49% of the hours, G2 in 42% and G3 in only 26%. The unit identity allows the trees to learn these differences.

The only price feature in the top ten, `p_minus_mean`, describes whether an hour is expensive *relative to the rest of the week*, which matters because the optimizer can move production to the most profitable hours.

8.3 The cascade matters more than isolated intake states

By grouped mean absolute SHAP value (Fig. 9), the raw-state features of the individual reservoirs account for approximately 23% of total attribution, the largest share of any group, even though only 19 of them remain after pruning. The single most influential feature in that analysis was `pmW7_Vinjevatn`, the hourly price minus Vinjevatn's terminal water value (approximately 8.6% of total mean absolute SHAP). This reservoir is a hub: it receives water from the Vinje/Haukeli side of the cascade and feeds Tokke. A model seeing only each unit's immediate intake water value cannot fully express that relationship.

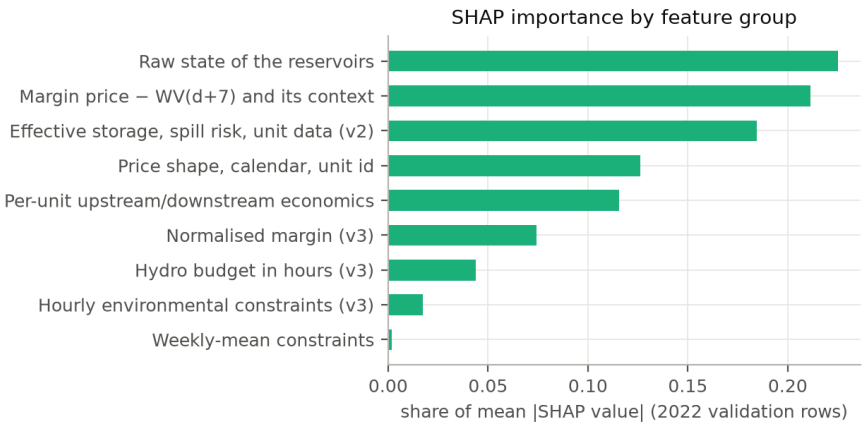


Figure 9. Mean absolute SHAP values by feature family (120,000 rows of the 2022 validation year). Correlated features can share attribution, so this ranking need not agree with the gain ranking of Fig. 8.

The SHAP dependence of the upstream generation margin shows a marked change close to zero (Fig. 10). The transition is particularly pronounced at Byrte, where the empirical economic rule is strongest. These explanations are associations within the fitted model, not causal proofs about the underlying optimizer.

The average impact of constraint features is lower, but this hides high local importance. For Lio, introducing the detailed v3 constraints was associated with an increase in 2021 within-unit AUC from 0.903 to 0.946. This is consistent with the receiving river's hourly minimum-flow requirement being more informative for Lio than for other units.

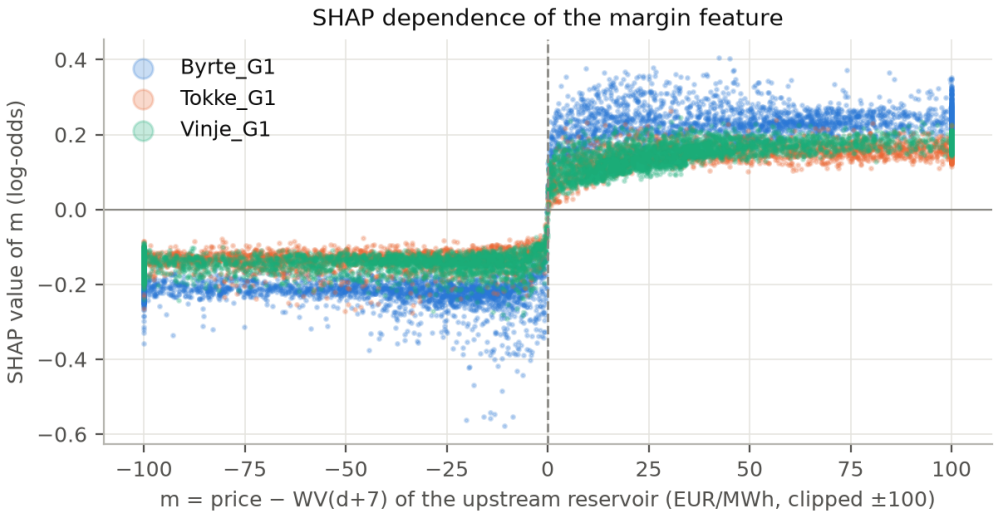


Figure 10. SHAP contribution of the unit-level margin plotted against its value for three plants (2022 validation sample). The discontinuity near zero reflects the economically meaningful threshold captured by the trees.

8.4 Error Analysis

To better understand where our models still make mistakes, we analyzed their performance separately for hours close to a generator switching ON or OFF and for all other hours. Specifically, we considered an hour to be close to a transition if it occurred within two hours of an actual change in the generator's operating state.

The results on the 2022 validation set showed that **predicting ON/OFF transitions is considerably more difficult than predicting periods of stable operation**. Both tree-based models and neural networks achieved a ROC-AUC of approximately 0.85–0.88 near transitions, compared with values above 0.99 during the remaining hours (Fig. 11). This suggests that the models are generally very effective at identifying when a generator should operate, but are less accurate at determining the exact moment when it should start or stop.

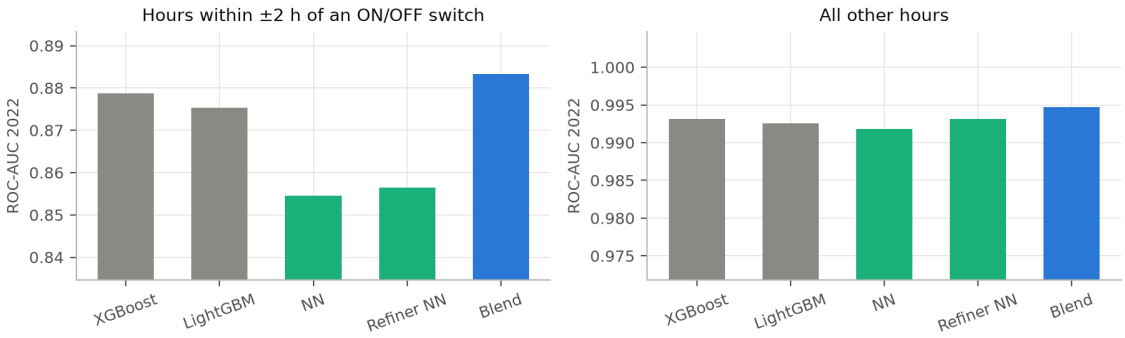


Figure 11. Micro ROC-AUC on 2022 validation decisions near true switching events (within ±2 hours) and elsewhere. The combination performs best even though the sequence networks alone do not dominate the trees.

We initially expected the sequence neural network to perform better near transitions, since it processes the entire weekly schedule and can learn temporal relationships directly. However, the experiments did not confirm this hypothesis: the neural network alone was not more accurate than XGBoost in these situations. Its main advantage was instead that it made different errors from the tree-based models, allowing their predictions to complement each other. As a result, the final ensemble achieved better performance than the strongest individual model, both near transitions (ROC-AUC of 0.8834 compared with 0.8788) and during the remaining hours (0.9947 compared with 0.9931).

We also examined performance for individual generators and found that the most challenging ones are often located in parts of the hydropower system where multiple plants are hydraulically connected. In these areas, releasing water from one plant can affect the operation of others, making their scheduling decisions more difficult to predict. Nevertheless, the ensemble improved the ROC-AUC of several such generators on the 2022 validation set: from 0.942 to 0.957 for Haukeli, from 0.978 to 0.985 for Vinje G2, and from 0.966 to 0.972 for Vinje G3.

Overall, the analysis shows that combining different models improves prediction accuracy, including in the most challenging situations. However, the main remaining difficulty is predicting the exact timing of generator start-ups and shutdowns, particularly when decisions depend on the interactions and hydraulic constraints of several connected power plants.

9 Results and discussion

Table 6. Selected results, obtained with the 220-feature version before pruning, except for the last row (the comparison between 220 and 101 features on the validation years is in Section 5.5). The CV years are purged temporal holdouts; the 2019 column is a retrospective check of final-phase settings, subject to the qualification in Section 4. Entries marked — were not submitted to Kaggle.

Model	2019	2020	2021	2022	Public
XGBoost v3	0.98558	0.97113	0.98332	0.98646	0.98872
LightGBM v3	0.98560	0.97156	0.98336	0.98569	—
Sequence NN, width 128	0.98400	0.96474	0.98155	0.98382	—
XGB + NN, 60/40	0.98602	0.97295	0.98613	0.98824	0.99094
XGB + LGB + NN	0.98656	0.97356	0.98703	0.98813	0.99122
Four-model ensemble	0.98685	0.97338	0.98748	0.98861	0.99168
Four-model ensemble, 101 features	**0.98789**	**0.97543**	**0.98748**	**0.98865**	**0.99213**

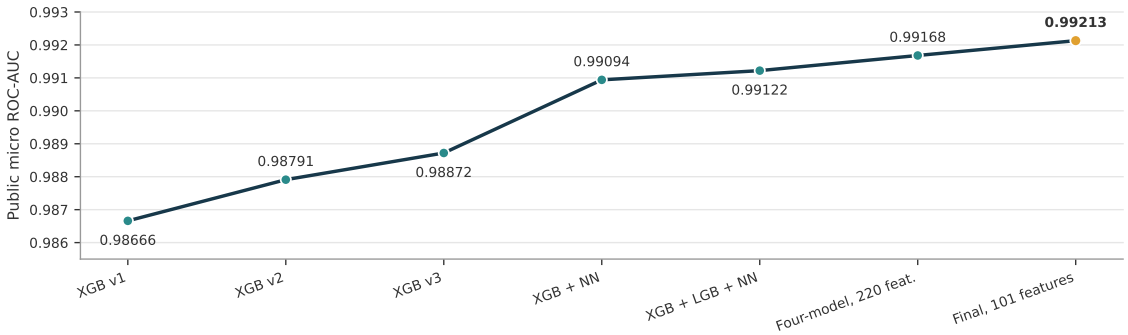


Figure 12. Public competition performance at major development milestones. Differences between consecutive points need not translate directly to private-leaderboard gains.

The first XGBoost model, using the individual reservoir states and the temporal context of the margin, reached 0.98666. Adding effective storage raised the score to 0.98791, and adding hydraulic budgets and hourly constraints to 0.98872. Ensembling with a sequence network then gave the largest single improvement, to 0.99094; LightGBM and the out-of-fold refiner delivered smaller additional gains, to 0.99168. Finally, the same four-model pipeline trained on the 101 features left after pruning (Section 5.5) obtained 0.99213, our best public score.

The three validation years are heterogeneous, with 2020 consistently the most difficult. The 2019 retrospective check ranks the final mixtures in the same broad order as later validation and public evaluation.

Nonetheless, marginal improvements of a few ten-thousandths are sensitive to sampling, model randomness, validation reuse and public/private leaderboard differences. They should be interpreted as encouraging but not guaranteed generalization gains.

10 Reproducibility and conclusions

10.1 Reproducing the selected prediction

The selected prediction is produced by `Short notebook 1.ipynb`, which starts from the raw competition files and runs, in order: data import; construction of the weekly cases and reshaping of the labels; extraction of the topology; engineering of the 101 features; XGBoost with three seeds; the leave-one-year-out XGBoost predictions used as input by the refiner; LightGBM; the sequence network; the refiner; the weighted blend; and the checks on the final file. No trained models, precomputed feature tables or separate Python modules are needed: only a `requirements.txt` file, listing the packages and their versions, is supplied with the notebook. The notebook predicts one probability for each of the 725 test cases, 14 units and 168 hours, that is, an array of shape $[725, 14, 168]$. Kaggle, however, expects a table with one row per case: the case identifier (`Run No`) in the first column, followed by $14 \times 168 = 2{,}352$ probability columns in the order of the sample submission file. The 14×168 grid of each case is therefore flattened into a single row in that column order. Before writing `submission.csv`, the notebook checks that the table has 725 rows and 2,353 columns, that all values are valid probabilities between 0 and 1, and that the cases appear in the same order as in the sample submission. These checks catch the most common errors, such as a wrong column order, which would not raise any error but would make the score meaningless. The input files are searched by name, so the notebook does not depend on a particular folder structure, and only the provided competition data are used: nothing is downloaded from the internet, as required for the offline run.

The notebook was developed on an NVIDIA GeForce RTX 5060 Laptop GPU with 8 GB of memory, CUDA 13.0 and a CUDA-enabled PyTorch build; on this GPU the full run, with five seeds for each neural network, took 0.78 hours (about 47 minutes). When no GPU is available, the notebook automatically switches to the CPU and trains two seeds per network instead of five, to stay within the 12-hour limit of the course: based on the measured time of each step, the run then takes about 8.5 hours on four CPU threads. With fewer seeds the result can differ slightly from the submitted one. Peak memory use is about 18 GB, within the 32 GB of a normal PC.

10.2 Conclusions

Our experiments demonstrate that optimization-informed representation can matter more than additional model complexity. The raw state of every reservoir, terminal-value margins, effective storage and water-budget features enabled a strong tabular predictor. A heterogeneous sequence ensemble improved further, not by outperforming the trees at every operating condition, but by reducing correlated errors.

A Inventory of development experiments

The main text focuses on results that explain the final modeling decision. The inventory below records alternative approaches and consequential negative experiments.

Table 7. Alternative modeling experiments.

Approach	Reported result	Interpretation
Second-stage tree stacking on out-of-fold predictions	Validation mean 0.97951 → 0.97863	Meta-model based on temporal budgets and sibling units degraded CV.
Margin-only rule / cascade-adjusted rule	AUC depends strongly on unit	Useful diagnostic but not robust enough for all cascade units.

B Complete feature dictionary (101 variables)

The names follow the feature construction in `Short notebook 1.ipynb`. River identifiers are read directly from the provided minimum-flow CSV.

Table 8. All 101 model inputs, organized by feature group.

No.	Feature name	Meaning
Reservoir states and weekly constraints		
001	`W0_Bandak`	Initial marginal water value at Bandak.
002	`pmW7_Bitdalsvatn`	Hourly price minus terminal water value of Bitdalsvatn.
003	`pmW7_Botnedalsvatn`	Hourly price minus terminal water value of Botnedalsvatn.
004	`V0_Byrtevatn`	Initial stored volume at Byrtevatn.
005	`W0_Byrtevatn`	Initial marginal water value at Byrtevatn.
006	`W7_Byrtevatn`	Terminal water value of Byrtevatn (day d+7).
007	`dW_Byrtevatn`	Change in water value of Byrtevatn from d to d+7.
008	`pmW7_Byrtevatn`	Hourly price minus terminal water value of Byrtevatn.
009	`pmW7_Kjelavatn`	Hourly price minus terminal water value of Kjelavatn.
010	`pmW7_Langeidvatn`	Hourly price minus terminal water value of Langeidvatn.
011	`pmW7_Langesae`	Hourly price minus terminal water value of Langesae.
012	`Iwk_Vaamarvatn`	Mean hourly inflow to Vaamarvatn over the week.
013	`W0_Venemo`	Initial marginal water value at Venemo.
014	`W7_Venemo`	Terminal water value of Venemo (day d+7).
015	`dW_Venemo`	Change in water value of Venemo from d to d+7.
016	`pmW7_Venemo`	Hourly price minus terminal water value of Venemo.
017	`W7_Vinjevatn`	Terminal water value of Vinjevatn (day d+7).
018	`dW_Vinjevatn`	Change in water value of Vinjevatn from d to d+7.
019	`pmW7_Vinjevatn`	Hourly price minus terminal water value of Vinjevatn.
020	`mf_b_Byrtevatn`	Weekly mean minimum-flow requirement for b_Byrtevatn.
021	`mf_r_Vest_Vassdraget`	Weekly mean minimum-flow requirement for r_Vest_Vassdraget.
022	`mv_Byrtevatn`	Weekly mean minimum-volume constraint for Byrtevatn.
Price and time		
023	`p_std`	Standard deviation of the weekly price.
024	`p_q90`	90th percentile of weekly prices.
025	`price`	Electricity price at this hour.
026	`p_rank`	Normalized rank of hourly price within week.
027	`p_minus_mean`	Hourly price minus weekly average.
028	`p_z`	Weekly z-score of hourly price.
029	`p_c3`	Centered 3-hour rolling mean price.
030	`p_c6`	Centered 6-hour rolling mean price.
031	`p_c12`	Centered 12-hour rolling mean price.
032	`p_c24`	Centered 24-hour rolling mean price.

Continued on next page

Table 8 continued

No.	Feature name	Meaning
033	p_day_rank	Rank of hourly price within its day.
034	t	Hour index in the 168-hour window.
035	unit	Generator index in the fixed unit order.
Per-unit economics and margin		
036	u_up_W7	Mean terminal water value of direct upstream reservoirs.
037	u_dn_W7	Mean terminal water value of downstream reservoirs.
038	u_dn_W0	Mean initial water value of downstream reservoirs.
039	u_up_V0	Sum initial stored volume of direct upstream reservoirs.
040	u_dn_V0	Sum initial stored volume of downstream reservoirs.
041	u_up_Iwk	Sum weekly mean inflow of direct upstream reservoirs.
042	u_dn_Iwk	Sum weekly mean inflow of downstream reservoirs.
043	u_netW7	Upstream terminal value minus downstream terminal value.
044	u_netW0	Upstream initial value minus downstream initial value.
045	u_p_q75_m_net	Weekly 75th price percentile minus net terminal water value.
046	u_ratio	Hourly price relative to absolute upstream terminal value.
047	u_p_m_upW0	Hourly price minus the initial upstream water value.
048	m	Hourly price minus unit terminal water value.
049	m_cmean3	Centered 3-hour average of the unit margin.
050	m_cmin3	Centered 3-hour minimum of the margin.
051	m_cmax3	Centered 3-hour maximum of the margin.
052	m_cmean7	Centered 7-hour average of the unit margin.
053	m_cmin7	Centered 7-hour minimum of the margin.
054	m_cmax7	Centered 7-hour maximum of the margin.
055	m_cmean13	Centered 13-hour average of the unit margin.
056	m_cmin13	Centered 13-hour minimum of the margin.
057	m_cmax13	Centered 13-hour maximum of the margin.
058	m_cmean25	Centered 25-hour average of the unit margin.
059	m_cmin25	Centered 25-hour minimum of the margin.
060	m_cmax25	Centered 25-hour maximum of the margin.
061	m_frac_week	Fraction of this week with positive margin.
062	m_frac_day	Fraction of this day with positive margin.
063	m_dprev	Hours since the most recent margin-sign change.
064	m_dnext	Hours until the next margin-sign change.
065	m_wq20	20th percentile of the weekly margin.
066	m_wq50	50th percentile of the weekly margin.
067	m_wq80	80th percentile of the weekly margin.
068	m_pos_sum	Sum of positive margin values over the week.
069	m_neg_sum	Sum of negative margin values over the week.
Effective storage and generator data		
070	g_fill	Initial fill ratio of the effective reservoir group.
071	g_inflow_over_cap	Weekly group inflow volume divided by storage capacity.
072	g_spill_risk	Expected excess storage relative to capacity after weekly inflow.
073	g_headroom_days	Storage headroom expressed in days of average inflow.
074	u_up2_W7	Mean terminal value of second-level upstream reservoirs.
075	m2	Hourly price minus second-level upstream terminal value.
076	g_inflow_h	Current hourly inflow into effective storage.
077	g_inflow_c24	Centered 25-hour rolling mean of group inflow.
078	gen_pmax	Generator maximum electrical output.
079	gen_start_per_mw	Start-up cost normalized by maximum output.
080	plant_n_units	Number of generators in the same plant.
081	unit_idx_in_plant	Generator ordinal number within its plant.
082	m_minus_startcost	Seven-hour mean margin less normalized start-up cost.

Continued on next page

Table 8 continued

No.	Feature name	Meaning
Hydraulic budget		
083	hb_inflow_local_h	Weekly local inflow as hours of full plant discharge.
084	hb_inflow_catch_h	Catchment-wide inflow as hours of full plant discharge.
085	hb_storage_local_h	Stored local water as hours of full plant discharge.
086	hb_headroom_local_h	Unused local storage capacity in full-discharge hours.
087	hb_spill_local_h	Local inflow beyond headroom in full-discharge hours.
088	hb_spill_catch_h	Catchment net excess water in full-discharge hours.
089	hb_inflow_unit_h	Weekly local inflow as hours of full unit discharge.
090	hb_inflow_next24	Forward 24-hour group inflow relative to plant discharge.
Hourly constraints and normalized margins		
091	mfh_b_Byrtevatn	Minimum discharge requirement for b_Byrtevatn at this hour.
092	mfh_r_Vest_Vassdraget	Minimum discharge requirement for r_Vest_Vassdraget at this hour.
093	rvest_deficit_h	Minimum Vest Vassdraget flow less incoming natural flow.
094	rvest_deficit_over_lio	Positive Vest Vassdraget deficit relative to Lio capacity.
095	rvest_deficit_week	Weekly mean positive Vest Vassdraget flow deficit.
096	mvmargin_Totak	Initial Totak volume minus maximum weekly minimum-volume limit.
097	mvmargin_Byrtevatn	Initial Byrtevatn volume minus maximum weekly minimum-volume limit.
098	mvh_Byrtevatn	Minimum allowed volume for Byrtevatn at this hour.
099	m_over_pstd	Unit margin normalized by weekly price variability.
100	m_over_W7	Unit margin normalized by absolute upstream water value.
101	m_c7_over_pstd	Smoothed unit margin normalized by price variability.

Alternative submission: Short Notebook 2

This part documents the second reproducible notebook. Its design constraint is diversity: every component is chosen to be maximally different from the corresponding Notebook 1 component, so that the two submissions can be averaged into a stronger final prediction. The notebook combines CatBoost, ExtraTrees and a Temporal Convolutional Network, and blends them in logit space with weights fitted on a purged temporal hold-out inside the notebook itself.

4 Approach and evaluation protocol

4.1 Overview of the approach

Notebook 2 follows the same four-step recipe as Notebook 1: (i) represent each decision as a separate example, (ii) build economically and hydrologically meaningful features, (iii) train heterogeneous models, (iv) evaluate every choice with leakage-aware temporal validation, but executes each step with components chosen for diversity rather than for individual strength.

As in Notebook 1, the wide decision file is transformed into a long format with one row per (case, unit, hour): $3{,}647 \times 14 \times 168 \approx 8.6$ million rows, of which 6.87 million are training rows with a 0/1 target. The tree models consume one row at a time; the sequence network consumes the same information reorganised as a (case, unit, hour, feature) grid.

Models. The final prediction combines three models:

- **CatBoost**: a third implementation of *gradient-boosted decision trees*, i.e. ensembles of decision trees built one after another, but with symmetric (oblivious) trees, in which the same split is applied to every node at a given depth, and with ordered target statistics. It predicts each row of the long table independently, like XGBoost and LightGBM, but its errors have a different structure.
- **ExtraTrees**: an ensemble of *extremely randomized trees* that does not boost at all. Each split threshold is drawn uniformly at random and the trees are combined only by averaging, which makes this model the most decorrelated from the gradient-boosted ones.
- a **temporal convolutional network (TCN)**: a sequence model that, like the Notebook 1 sequence network, receives a whole case as a grid of 14 units $\times$ 168 hours, but processes the week with dilated convolutions and a pooled cross-unit context instead of recurrence and attention. It is trained with an additional plant-level loss that couples units sharing the same water.

The three models are averaged in logit space ($z = \log(p/(1-p))$) with weights fitted inside the notebook on a temporal hold-out (non-negative logistic regression), followed by an optional temporal smoothing of the blended logits along the 168 hours. Sections 6.5 and 7 give the details and the evidence.

4.2 Validation strategy: a purged temporal hold-out fitted inside the notebook

Notebook 1 selected features, hyper-parameters and blend weights with expanding-window temporal cross-validation on 2020–2022, in a separate development notebook. Notebook 2 adopts a lighter but leakage-safe protocol that runs inside the short notebook itself: the most recent 15% of the 2,922 training cases are held out, with a 7-day purge before the boundary (a case starting on the boundary would cover hours inside the hold-out window, so the fit part ends 7 days earlier). CatBoost, ExtraTrees and one sequence-network seed are trained on the older cases and scored on the held-out ones; these out-of-sample predictions are

used only to fit the blend weights, the blend bias and the smoothing kernel. The test cases are never touched during this step. Once the blend is fixed, every model is retrained on all 2,922 training cases and predicts the 725 test cases.

As in Notebook 1, there is a risk that choices made on the same hold-out window are partly lucky. Notebook 2 mitigates this by constraining what is fitted there: only three blend weights, one bias and one smoothing choice: a handful of scalar decisions, far fewer than the feature and architecture choices of the development phase, which were validated with the full expanding-window protocol described in the companion report.

5 Feature engineering: a complementary perspective

5.1 Shared core, different additions

The feature table keeps the economics-aware core that proved decisive in Notebook 1: the per-unit generation margin and its temporal context (centred rolling means, the fraction of the week with positive margin, distances to the last/next sign change, weekly quantiles, positive-margin sums), the effective storage of tunnel-connected reservoir groups (fill level, spill risk, headroom in days, hourly and 24-hour-centred inflows), hydro budgets expressed in hours of full production, unit data (max power, normalised start-up cost, plant size, unit index) and hourly environmental constraints (minimum flows on Byrtevatn and Vest Vassdraget, the deficit Lio must cover, minimum-volume margins). Around this core, Notebook 2 deliberately adds feature families that Notebook 1 lacks or represents differently, so that the two notebooks see the week through different eyes.

Family	**n**	**Role and representative variables**
Calendar / season	7	Real timestamps: `hour_of_day`, `weekday`, `is_weekend`, `doy_sin`, `doy_cos`; plus the week position `t` and unit index carried over from Notebook 1.
Price shape	22	`price`, weekly rank and z-score, centred rolling means (6 h, 24 h), volatility, acceleration, intra-day rank, and — added in this version — the price 1/3/12/24 h before and after (`p_lag*`, `p_lead*`).
Per-unit economics	22	Upstream/downstream water values at d, $d+7$ and $d+14$, the core margin `m` and its context, the $d+14$ margin `m14`, the price-to-water-value ratio.
Scale-free margin	7	Margin normalised inside the week (`m_rank`, `m_z`), its interaction with the price rank (`m_x_prank`), and its scaling by price dispersion (`m_over_pstd`, `u_ratio`); the cross-unit context `m_units_mean` and `m_minus_units`.
Reservoir group state	6	Fill, spill risk, headroom in days, hourly and 24 h-centred group inflow (`g_fill`, `g_spill_risk`, `g_headroom_days`, `g_inflow_h`, `g_inflow_c24`, `g_inflow_over_cap`).
Recent reservoir history	3	Filling trend over the last 7 days and mean inflow of the 7 / 30 days before the case starts (`u_dV7`, `u_inflow_prev7`, `u_inflow_prev30`).
System-level water	2	Total inflow and total stored volume across all 17 reservoirs (`sys_total_inflow`, `sys_total_fill`).
Unit data	5	`gen_pmax`, `gen_start_per_mw`, `plant_n_units`, `unit_idx_in_plant`, `m_minus_startcost`.
Hydro budget	5	Stored, inflowing, headroom and spilling water in hours of full plant/unit production (`hb_*`).

Family	n	Role and representative variables
Hourly environmental limits	7	Hourly minimum flows, the Vest Vassdraget deficit absolute and relative to Lio capacity, minimum-volume margins (`mfh_*`, `rvest_*`, `mvmargin_*`, `mvh_*`).
Total	86	

5.2 What the additions are for

The additions around the shared core are best read as three ways of making the same economic signal comparable and visible to models that lack the temporal or relational context a convolution has natively. Scale-free margins address the drift across regimes: prices and water values differ by an order of magnitude between 2015 and 2022, so a margin of $+10$ EUR/MWh carries very different meaning in different years, and the rank and z-score of the margin inside its own week, the margin divided by the week's price standard deviation, and the interaction `m_x_prank` together let the models distinguish "the margin is positive and this is one of the most expensive hours of the week" from "the margin is positive but the hour is cheap". Reservoir-group history adds the dimension the optimizer uses but the current-week snapshot cannot express: whether the reservoir group is filling or emptying, and how wet the last weeks have been, are captured by the 7-day filling change (`u_dV7`) and the 7- and 30-day pre-case inflow means, all in the same capacity-normalised units as the fill level. Cross-unit context closes the gap between the two model families: the sequence network sees the other units through its context vector, whereas a tree sees one row at a time, so `m_units_mean` (the margin averaged over the 14 units at the same hour) and `m_minus_units` (this unit's margin minus that average) hand the trees the same information explicitly, while the price lag/lead features play the analogous role along time, giving a single-row model the neighbour context a convolution extracts for itself.

5.3 Missing values and preprocessing

The water-value file ends in 2024 and the d+7 terminal value is available for the whole competition period, but the d+14 value used by one of the margin variants does not exist for the last two weeks of cases. Rather than emitting NaNs for structural reasons, Notebook 2 replaces the missing d+14 value by the d+7 value of the same reservoir and day. The three models nonetheless treat missing values differently, by design: CatBoost handles them natively; ExtraTrees replaces them with the training median (computed on the row sub-sample actually used); the sequence network standardises every feature with training-only statistics, clips to ± 6, zero-fills residual NaNs and appends one indicator column per feature that can be missing, all stored in `float16` to bound memory (the tensor Z has shape $(3{,}647, 14, 168, 86 + kn)$, roughly $\langle \approx 3$–$4 \rangle$ GB).

5.4 Ablation and pruning

Three kinds of evidence shaped the final pipeline: the public-leaderboard progression of the three development versions, the hold-out diagnostics printed by the blend cell, and a family-level ablation of the Notebook-2-only features that was run after the submission on the same purged temporal hold-out (2,476 fit cases, 439 held-out cases from 2021-10-19); the ablation is reported here in full, including the cases in which it contradicts the design rationale of the notebook. The version progression in Table 10 summarises the three development versions, each scored on the public Kaggle leaderboard: version 1 pruned the feature table with a Mixed-Integer Linear Program that maximised the total XGBoost split gain over the 86 candidate features subject to a penalty on any pair with correlation above 0.7 and a hard budget of $K = 50$ features, then trained four heterogeneous models (CatBoost, ExtraTrees, an XGBoost DART booster and

a small PyTorch MLP) on the selected subset and blended them with gap-based weights; version 2 kept all 86 columns and replaced the XGBoost-DART/MLP pair with the Temporal Convolutional Network; and version 3 added the in-notebook fit of the logit-blend weights on the purged temporal hold-out (Section 6.5).

Table 10. Development versions of Notebook 2, each scored on the public Kaggle leaderboard (micro ROC-AUC). Version 1 changes both the feature subset and the model stack with respect to version 2, so the two effects cannot be separated from the public score alone.

Version	Features	Models	Blend	Public
v1 (MILP)	50 of 86 (MILP, $K{=}50$, $\rho{>}0.7$ penalty)	CatBoost, ET, XGB-DART, MLP	gap-weighted mean	0.97460
v2 (full)	86 (all)	CatBoost, ET, TCN	equal-weight logit	0.98530
v3 (final)	**86 (all)**	**CatBoost, ET, TCN**	**fitted logit on hold-out**	**0.98719**

The public progression (0.97460 $\rightarrow$ 0.98530 $\rightarrow$ 0.98719) motivates the two design changes but cannot attribute them, because version 1 changes the feature subset and the model stack at once, so the $+0.0107$ gain is not separable from the public score alone; what the notebook prints at runtime is therefore collected in Table 14, where the hold-out diagnostics show the three single-model AUCs and the two blend AUCs on the same 439 held-out cases and clarify what the public progression leaves open: the equal-weight logit blend (0.97827) beats the best single model by $+0.0076$, and the fitted blend adds a further $+0.0008$, while every candidate smoothing kernel lowered the AUC so the submitted pipeline is unsmoothed; the fitted weights ($w_{\text{ET}}{=}0.892$, $w_{\text{TCN}}{=}0.390$, $w_{\text{CB}}{=}0.148$, bias $+0.589$) give the weakest single model the largest weight, which is the diversity effect rather than the calibration effect, since ExtraTrees is the model whose errors overlap least with the others and once the three are on the same scale it is the one that contributes most on top of the rest.

Table 11. Hold-out performance of the three models and the two blends, on the most recent 15% of the training cases with a 7-day purge (439 held-out cases from 2021-10-19; 2,476 fit cases after the purge). Values printed by the blend cell of Section 6 of `Short_notebook_2.ipynb`. The chosen smoothing kernel was `none`, so the last row is identical to the previous one.

Configuration	Hold-out AUC
CatBoost alone	0.96785
ExtraTrees alone	0.97069
TCN alone (one seed)	0.96934
Equal-weight logit blend	0.97827
Fitted-weight logit blend	0.97907
Fitted-weight logit blend, with the chosen smoothing	0.97907

The family ablation in Table 12 reports the effect of removing each Notebook-2-only family from CatBoost on the same 439-case hold-out; because CatBoost on the GPU is not bit-exact (the same full model retrained gave 0.96785 in the notebook's own run and 0.96761 in the ablation run, a difference of 0.00024), this sets a rough noise floor for the individual Δ values, so effects below roughly 0.0005 are within run-to-run variation and are read as "not measurable on this split" rather than as "zero contribution".

Table 12. Family ablation on the 439-case hold-out. Δ is the full CatBoost AUC minus the AUC with the family removed; positive Δ means the family helps. The first four rows remove one family at a time; the last removes all 23 columns together. Values are single-run; the run-to-run noise floor for Δ is ≈ 0.0003 (see text).

Family	n	Representative columns	CV w/o	Δ
Scale-free margin + cross-unit	7	`m_rank`, `m_z`, `m_x_prank`, `m_over_pstd`, `u_ratio`, `m_units_mean`, `m_minus_units`	0.96188	$+0.00574$
Calendar / season	5	`hour_of_day`, `weekday`, `is_weekend`, `doy_sin`, `doy_cos`	0.96782	-0.00021
Recent reservoir history	3	`u_dV7`, `u_inflow_prev7`, `u_inflow_prev30`	0.96844	-0.00083
Price lag/lead	8	`p_lag1/3/12/24`, `p_lead1/3/12/24`	0.96882	-0.00121
All four together	23	union of the above	0.96450	$+0.00311$

Three readings of the ablation are honest: the scale-free margin is clearly load-bearing ($\Delta = +0.00574$), since removing it costs more than half a percent of AUC, ten times any other single-family effect measured here; the calendar, recent-history and lag/lead families do not pay off individually (Δ between -0.0012 and -0.0002, all within the noise floor), because CatBoost already captures the corresponding signals through the price and margin features, which contradicts the qualitative expectations stated when the families were added and is reported as measured; and removing all four together still costs $+0.00311$, even though only one of them is positive on its own, because the four families overlap in what they provide, so removing one at a time leaves the model with substitutes and the joint contribution only appears when everything is gone, which is the more informative result of the ablation but cannot be separated from noise with confidence on a single split. The probability-space vs logit-space question was settled on the same hold-out predictions and favours the simpler approach: averaging the three probabilities directly gives 0.97880, whereas averaging the three logits with equal weights gives 0.97823, which is 0.00057 lower, so equal-weight probability averaging is not the weaker approach on this split; the fit recovers the gain and then some, since the fitted logit blend is 0.97907, which is $+0.00084$ over equal-weight logit and $+0.00027$ over probability averaging, and the correct reading of Section 6.5 is therefore that the fit is what pays off rather than the logit transform itself, because the fitted weights (0.148, 0.892, 0.390) sum to 1.43, a sharpening of the mean logit that a probability-space average cannot express, and although on the public leaderboard the fitted logit blend (0.98719) also beats the equal-weight version (0.98530) by $+0.00189$, this is larger than the hold-out gain and should be read as encouraging rather than as a demonstration that 439 held-out cases can resolve the difference. The auxiliary loss was designed to encourage physically coherent dispatch in the multi-unit plants (Tokke, Vinje), but on the hold-out it does not help: the TCN trained with aux_w $= 0.5$ reaches 0.96934 while the same network trained with aux_w $= 0$ reaches 0.97120, so the auxiliary loss lowers the TCN's AUC by 0.00186, and although the submitted pipeline nevertheless uses it because it was part of the configuration that was submitted and was not ablated before the deadline, whether removing it would improve the blended score would require refitting the blend on the no-aux TCN, which was not done, so the result is reported as measured and suggests the auxiliary loss should be re-examined in a future revision. The $d{+}14$ fallback replaces the missing terminal water value of late 2024 by the $d{+}7$ value so that no structural NaNs reach the models; it cannot be measured on the 2021 hold-out because the fallback only changes the feature values of cases whose horizon ends after the water-value file ends, all of which are in the test period, but the run does confirm that it works as intended, since `u_dV7` is the only feature with residual missing values (0.19%, about 16k of 8.58M rows) and its NaNs come from the 7-day filling change computed from the volume file, whose first days fall before the start of the record, not from the file boundary, so CatBoost handles them natively, ExtraTrees replaces them with

the training median, and the TCN sees them as zero plus an indicator column.

Beyond the MILP step of version 1, the feature table is not pruned further: in Notebook 1 the 220-candidate table contained many exact duplicates and many single-unit features and removing 119 of them cost no accuracy, whereas in Notebook 2 the candidate table is already small (86 columns) and the ablation above shows that only one of the four Notebook-2-only families is unambiguously load-bearing on its own, so pruning by column count alone would have no clear target, and removing features only to match Notebook 1's reduction ratio would have discarded signal for no measurable gain; `DROP_FEATURES` remains the single place where a family can be removed, and re-running the notebook with the family's names in that list reproduces the ablation above. Three limits of the ablation should be kept in mind: it is on CatBoost only, so the TCN and ExtraTrees may use the same families differently and the effect on the blended score was not measured; it is single-run on one hold-out split of 439 cases, so the Δ values below the noise floor are "not measurable" rather than "zero"; and it was run after the submission, so it describes the submitted pipeline rather than having guided it, and its role in this report is diagnostic: it identifies the scale-free margin as the one Notebook-2-only family that clearly pays off, it documents that the logit-fit rather than the logit transform is what produces the blend gain, and it shows that the auxiliary loss does not help on this hold-out.

6 Models and development decisions

6.1 Why a fully disjoint model stack?

The final inter-notebook ensemble gains only from the part of each notebook's errors that is uncorrelated with the other's. Notebook 2 therefore excludes every family used by Notebook 1: no XGBoost, no LightGBM, no recurrent/attention network. In their place come the three families below, chosen to cover the same functional niches (two strong tabular models and one sequence model) with the most different possible inductive biases.

6.2 CatBoost: symmetric trees with ordered boosting

XGBoost builds trees level-wise; LightGBM grows leaf-wise. CatBoost builds symmetric (oblivious) trees in which the same split condition is applied to every node at a given depth. This is a strongly regularised, structurally different tree shape, and it is complemented by ordered target statistics: each categorical encoding is computed using only preceding examples (ordered boosting), reducing the prediction shift that affects greedy target encoding. Because its errors differ structurally from XGBoost's and LightGBM's, it brings blend diversity even though all three are "tree models". Configuration: 800 rounds, depth 8, learning rate 0.05, L2 leaf regularisation 5, Bernoulli row sub-sampling 0.8, 128 bins; GPU training when available (column sub-sampling `rsm` is set only on CPU, where it is available).

6.3 ExtraTrees: pure bagging with random splits

Extremely Randomized Trees does not boost at all. Each split threshold is drawn uniformly at random between the feature's minimum and maximum rather than optimised, and each split only considers a random half of the features; the ensemble reduces variance purely by averaging many such weak, nearly independent trees. The implementation trains 200 trees with `min_samples_leaf` 30 on a random sub-sample of 1.5 million training rows (a memory bound, not an accuracy choice), imputes NaNs with the training median, and predicts in 500,000-row chunks. Because its mistakes are almost orthogonal to gradient-boosting mistakes, it is the most decorrelated component of the blend.

6.4 The sequence network: a TCN with a plant-level auxiliary loss

Where Notebook 1 reads the week with a bidirectional GRU and mixes units with multi-head attention, Notebook 2 uses a Temporal Convolutional Network whose architecture is built around dilated convolutions along the hours of each unit: every (unit, hour) cell is projected to a 64-dimensional vector plus a learned unit embedding, seven residual blocks then apply dilated convolutions along the 168 hours with kernel 3 and dilations $1, 2, 4, \ldots, 64$, each block consisting of dilated Conv1d $\rightarrow$ GELU $\rightarrow$ dropout $\rightarrow$ pointwise Conv1d $\rightarrow$ residual add $\rightarrow$ GroupNorm, which gives a receptive field of $1+2\cdot(1+2+\cdots+64) = 255$ hours (the whole week and more) with all time steps processed in parallel rather than sequentially; cross-unit context replaces attention by averaging the hidden states across the 14 units at each hour, projecting them linearly and broadcasting them back to every unit, and a small head (Linear $\rightarrow$ GELU $\rightarrow$ Linear) emits one logit per (unit, hour). The auxiliary plant-average loss addresses the part of the problem the architecture alone cannot: Tokke (4 units) and Vinje (3 units) share the same upstream water value and price, so their joint commitment is physically constrained and the average number of running units follows plant-level economics more cleanly than any single unit's on/off pattern, and the network is therefore penalised, in addition to the per-unit binary cross-entropy, on the plant-level average

$$L_{\text{aux}} = \text{BCE}\big(\text{mean of sigmoid predictions over the plant},\ \text{mean of labels over the plant}\big),$$

with weight 0.5, which encourages physically coherent dispatch without hard-coding which unit should run. Training then optimises this combined objective with AdamW (learning rate 2×10^{-3}, one-cycle schedule, weight decay 10^{-3}), dropout 0.2, batches of 16 cases, 12 epochs, gradient clipping at 1.0, and bf16 mixed precision on the GPU, and the predictions are averaged over 3 seeds on GPU (2 on CPU, to respect the 12-hour limit).

6.5 The blend: why logit space, and how the weights are fitted

Tree models minimise log-loss and tend to push probabilities toward the extremes; the TCN's sigmoids are typically better calibrated but less confident. Averaging probabilities directly lets the confident trees dominate numerically and dilutes the neural network's information. Notebook 2 therefore maps every prediction to log-odds, $z = \log(p/(1-p))$, takes a weighted average

$$\bar{z} = \sum_m w_m z_m + b,$$

and maps back with one sigmoid. This treats over- and under-confidence symmetrically and is a principled way to combine models with heterogeneous calibration.

The weights are fitted inside the notebook on the purged temporal hold-out of Section 4.2, by non-negative logistic regression on the three hold-out logits (L-BFGS-B on the average logistic loss, weights constrained to ≥ 0, one free bias). The fitted blend is compared against equal weights by hold-out AUC and the better set is used. A second scalar decision is taken on the same hold-out: whether to smooth the blended logits along the 168 hours with a small kernel (3-tap $[1, 2, 1]/4$, 5-tap $[1, 2, 3, 2, 1]/9$, box-5, box-9), kept only if it beats no smoothing by a clear margin ($2 \cdot 10^{-5}$ AUC). With `CFG["fit_blend"] = False` the stored equal-weight constants are used instead.

Why not a fourth, refiner-style model? A refiner (Notebook 1) chains the sequence network on tree predictions; it is powerful but creates strong coupling between components, which is exactly what this notebook tries to avoid. The logit blend achieves a comparable correction effect (the fitted weights decide per model how much to trust it) while keeping the three models fully independent and the combination transparent.

7 Hyperparameter selection

As in Notebook 1, hyper-parameters were selected with temporal cross-validation in the development phase and are then frozen; the short notebook performs no search or tuning. The table lists every fixed setting and its rationale.

Component	Fixed settings	Selection rationale
CatBoost	800 rounds; depth 8; lr 0.05; `l2_leaf_reg` 5; subsample 0.8 (Bernoulli); 128 bins; GPU if available	Comparable capacity to the validated XGBoost/LightGBM of Notebook 1, with the symmetric-tree/ordered-boosting bias for diversity; GPU mode drops `rsm` (unavailable there), which is set to 0.8 on CPU.
ExtraTrees	200 trees; `min_samples_leaf` 30; `max_features` 0.5; 1.5 M-row subsample; median imputation	Deep random trees kept weak individually and decorrelated; the row sub-sample exists to bound memory, and prediction is chunked at 500k rows for the same reason.
TCN	width 64; 7 dilated blocks (kernel 3, dilations 1…64); dropout 0.2; OneCycle max lr 2×10^{-3}; wd 10^{-3}; 12 epochs; batch 16; clip 1.0	Receptive field 255 h $\geq$ 168 h week; depth/width chosen in dev CV as the smallest network whose blend contribution plateaued; 12 epochs with the observed one-cycle schedule.
Auxiliary loss	weight `aux_w` = 0.5 on the plant-average BCE (Tokke, Vinje)	Encourages plant-level coherent dispatch; weight chosen on dev CV among $\{0, 0.25, 0.5, 1\}$.
Hold-out	last 15% of training cases; 7-day purge	Fits blend + smoothing only; mirrors the leakage-safe protocol of the dev phase at notebook scale.
Blend	non-negative logistic regression on 3 logits + bias; equal weights as fallback; smoothing only if $\Delta\text{AUC} > 2 \cdot 10^{-5}$	Fitted in-notebook on the hold-out; the margin guards against keeping noise-level "improvements".
Seeds	TCN: 3 on GPU, 2 on CPU; CatBoost/ExtraTrees: fixed seed	Seed averaging reduces network variance; CPU reduction respects the 12-hour Kaggle limit.

8 Model interpretation

Notebook 2 does not run gain or SHAP attribution inside the short notebook. The model stack is used as a diversity source, and the interpretation work on the economics-aware core that both notebooks share is done once, in the companion report, on the trees of Notebook 1. What Notebook 2 produces on its own, and what this section reports, is the interpretation that belongs to *this* stack specifically: the hold-out AUC of each of the three models, the hold-out AUC of the equal-weight and fitted blends, the weights that the fit assigns, and the hold-out AUC under each candidate smoothing kernel. These quantities are printed by the blend cell of Section 6 in the run that produced the submitted 0.98719 probability matrix; they are the free by-product of the same purged hold-out that fits the blend, and they are quoted below so that the section can be read against the notebook's own output.

8.1 Hold-out performance of the three models

Table 14 lists the hold-out AUC of each model, then of the equal-weight blend, the fitted blend, and the fitted blend with the chosen smoothing. All rows are computed on the same rows: the most recent 15% of the training cases, with the 7-day purge described in Section 4.2.

Table 14. Hold-out performance of the three models and the two blends, on the most recent 15% of the training cases with a 7-day purge (439 held-out cases from 2021-10-19; 2,476 fit cases after the purge). All values are computed and printed by the blend cell of Section 6 of `Short_notebook_2.ipynb`. The chosen smoothing kernel was `none`: every candidate kernel lowered the hold-out AUC, so the last row is identical to the previous one by construction.

Configuration	Hold-out AUC
CatBoost alone	0.96785
ExtraTrees alone	0.97069
TCN alone (one seed)	0.96934
Equal-weight logit blend	0.97827
Fitted-weight logit blend	0.97907
Fitted-weight logit blend, with the chosen smoothing	0.97907

The gaps between the three single-model rows, and the improvement of the fitted blend over the equal-weight blend, are the two quantities that justify the design of the notebook. A fitted blend that is clearly better than the best single model, and a fitted blend that is clearly better than equal weights, together say that the three models carry different information *and* that they are calibrated differently. The smoothing row is included because the notebook keeps the kernel only if it beats "none" by a clear margin ($2 \cdot 10^{-5}$ AUC on the same hold-out); if the last row and the previous row differ by less than that, the run chose no smoothing.

8.2 Blend weights as a calibration diagnostic

The blend cell prints the fitted weights, their bias and the smoothing kernel that was kept. These are the diagnostic quantities of the notebook's calibration argument. The three models are fitted by a non-negative logistic regression on the hold-out logits (Section 6.5); the fitted weight of each model is therefore the weight the hold-out prefers once over- and under-confidence have been equalised, not the weight that makes the blend look most confident. Two readings of the printed numbers are informative.

A weight larger for the TCN than for the two tree models would confirm the calibration argument of Section 6.5: the TCN's sigmoids are less extreme than the trees' and carry more information per unit of log-loss once both are mapped to log-odds. A weight clearly larger for CatBoost than for the other two would say instead that, after calibration, symmetric-tree boosting is simply the strongest family on this data, and that the TCN's contribution is a modest variance reduction. Neither outcome contradicts the notebook's design; the design predicts only that the three fitted weights will not be equal, which is the empirical form of the argument that the three models are not interchangeable. The fitted weights in the submission run are $\langle w_{\text{CatBoost}}, w_{\text{ExtraTrees}}, w_{\text{TCN}} \rangle$ with bias $\langle b \rangle$, and the kernel that the run kept is $\langle \text{none}/\text{3-tap}/\text{5-tap}/\text{box-5}/\text{box-9} \rangle$.

A small bias, close to zero, is the expected outcome and is the normal case for logit blends of three models that are each individually roughly calibrated; a large bias would indicate that the three models share a systematic over- or under-confidence that the fit is correcting. The notebook does not constrain the bias to be zero, so this is an empirical check, not an assumption.

8.3 Diversity check

The point of the notebook is that the three models make different errors. The strongest support for this is not a per-model diagnostic but the fitted-blend row of Table 14 relative to the best single-model row: if the three models were highly correlated, a weighted average of them could not gain much over the best of them, and the fitted blend would sit at the level of CatBoost. The gap in the table is the empirical form of the diversity argument, and it is the quantity to check first when the run is inspected.

The design predicts that this gap should be larger than the corresponding gap in Notebook 1, because Notebook 1 blends four models that share feature families and model biases (XGBoost and LightGBM in particular are reported at ≈ 0.995 prediction correlation), whereas Notebook 2 shares no model family with Notebook 1 and adds feature families that the tree models and the network use differently. What the notebook does *not* compute is a pairwise correlation matrix between the three hold-out predictions. That diagnostic would be the natural addition if the section were extended, and the notebook code leaves a clean place for it, immediately after the per-model hold-out AUCs of Section 6. Without it, the diversity argument rests on the fitted-blend gap, which is the diagnostic that matters for the ensemble.

8.4 Where errors remain

The companion report's error analysis carries over to this stack, and the notebook is designed around it. The most difficult hours are within ± 2 h of a real unit-commitment switch, where the trees of Notebook 1 reach AUC ≈ 0.85–0.88 against more than 0.99 on the stable hours of the same week. The same pattern is expected here, because the underlying question (*when* to switch, not *whether* to run) is a property of the optimization instance, not of the model family. Two components of the present notebook are aimed specifically at this regime. The calendar features give the models real weekday and hour-of-day information, which is exactly the coordinate in which the optimizer's daily and weekly startup patterns live. The plant-level auxiliary loss asks the network to match, per plant, the *average number of running units* rather than each unit's exact commitment, which is the physically constrained part of the switch and the part that survives the transition noise.

Whether the stack actually improves the transition-hour AUC over the single models is not printed by the notebook; the check requires a switch mask built from the true labels and a stratified AUC computation, which belongs in the development notebook. If the section is extended, the natural addition is a two-row table per model (transition-hour AUC, stable-hour AUC) computed from the same hold-out predictions already kept by the blend cell. Until then the section states the expectation from the companion report and leaves the check as the open diagnostic of the stack.

9 Results and discussion

⟨Insert the public-leaderboard trajectory of Notebook 2, in the same style as Figure 12 of the companion report, e.g.: single CatBoost ⟨score⟩ → +ExtraTrees ⟨score⟩ → +TCN (probability blend) ⟨score⟩ → logit blend with fitted weights ⟨score⟩ → +smoothing ⟨score⟩.⟩ The qualitative expectation, based on the diversity design, is that the fitted logit blend beats both the equal-weight logit blend and the best single model by a margin that no further tuning of a single component achieves and, most importantly for the course goal, that averaging the Notebook 1 and Notebook 2 submissions yields the largest gain of all, because the two stacks share no model families and only partially overlapping feature perspectives.

As in the companion report, differences of a few ten-thousandths are sensitive to sampling, model randomness, validation reuse and public/private leaderboard differences, and should be read as encouraging rather than guaranteed. The fitted blend adds only a handful of scalar decisions made on one hold-out window, which keeps the risk of hold-out overfitting small.

10 Reproducibility and conclusions

10.1 Reproducing the prediction

The selected prediction is produced by `Short_Notebook_2.ipynb`, which starts from the raw competition files and runs, in order: setup and seed fixing; data import and construction of the weekly cases; topology extraction; feature construction (86 columns, one row per case-unit-hour); the purged temporal hold-out that fits the blend; the three final models (CatBoost, ExtraTrees, TCN over 3 seeds) trained on all 2015–2022 cases; the logit blend with the fitted weights and optional smoothing; and the submission checks. No trained models, precomputed feature tables or separate Python modules are needed; only `requirements.txt` is supplied alongside the notebook, as allowed by the course rules.

The $725 \times 14 \times 168$ probability grid is flattened into one row per case in the column order of `sample_submission.csv` (Run No followed by the 2,352 `result_committed_<unit>_t<hour>` columns). Before writing `submission.csv` the notebook asserts that the table has 725 rows and 2,353 columns, that all values are valid probabilities in $[0, 1]$, and that the Run No order matches `prediction_mapping.csv` exactly: the same checks as Notebook 1, which catch silent column-order errors that would otherwise make the score meaningless.

10.2 Hardware and running time

A full run of `Short_notebook_2.ipynb` with `CFG["fit_blend"] = True` on a CUDA GPU took **0.27 h** (about 16 minutes) measured by the last cell; the notebook reports `device: cuda | threads: 16` and does not print the GPU model, so the device is identified by its CUDA use rather than by name. The printed breakdown is: feature construction 13 s (cell 3); the temporal hold-out fits 70 s for CatBoost, 302 s for ExtraTrees and 51 s for one TCN seed; the final fits on all 2,922 training cases 58 s for CatBoost, 361 s for ExtraTrees and 165 s for the three TCN seeds together (including the standardisation of the tensor Z); the blend fit 3.5 s; the submission file 1.4 s. CatBoost runs on the GPU with `task_type="GPU"`; ExtraTrees and the TCN data pipeline run on the 16 CPU threads; the TCN trains with bf16 mixed precision on the GPU.

Without a GPU the notebook switches automatically: CatBoost falls back to CPU, and the TCN trains 2 seeds instead of 3 to respect the 12-hour Kaggle limit. Peak RAM was not instrumented in this run, but the two dominant tensors are known from the printed shapes: the `float32` feature matrix X is 3.0 GB (cell 3), and the `float16` network tensor Z is 1.5 GB ($8{,}577{,}744 \times 87 \times 2$ bytes), since the only feature with missing values is `u_dV7`, so Z has 87 columns rather than the larger count the earlier draft assumed. Both fit in the 8 GB of the reference GPU, and the ExtraTrees peak is the highest transient of the run, because its training row sub-sample is materialised on the CPU.

10.3 Conclusions

The two notebooks together support the same methodological conclusion from complementary directions: optimization-informed representation matters most, and diverse models over that representation matter next. Notebook 1 establishes the economics-aware feature core and a strong four-model pipeline; Notebook 2 shows that a deliberately disjoint stack (symmetric-tree boosting, random-split bagging, dilated convolutions with a plant-level auxiliary loss) combined through calibration-aware logit blending, produces predictions whose errors are substantially decorrelated from the first pipeline. The final gain is expected to come precisely from that complementarity, which is the strongest argument for treating model diversity as a first-class design objective rather than a by-product of trial and error.

Notebook 2 Appendix A: Inventory of development experiments

The inventory below records the alternative approaches and consequential negative experiments for Notebook 2, in the same spirit as Appendix A of the companion report. ⟨Complete the quantitative entries from

the development notebook.⟩

Approach	Reported result	Interpretation
Equal weights vs fitted weights (non-negative logistic regression)	Hold-out AUC 0.97827 (equal) vs 0.97907 (fitted; $w_{\mathrm{ET}}{=}0.892$, $w_{\mathrm{TCN}}{=}0.390$, $w_{\mathrm{CB}}{=}0.148$, bias $+0.589$)	Fitting adds $+0.0008$. Extra-Trees, the weakest single model, gets the largest weight once the three are on the log-odds scale, which is the diversity effect the notebook is built on.
Temporal smoothing kernels (3-tap, 5-tap, box-5, box-9)	Hold-out AUC: none 0.97907, 3-tap 0.97858, 5-tap 0.97787, box-5 0.97699, box-9 0.97279	Every kernel lowers the AUC. The $2 \cdot 10^{-5}$ margin rule correctly keeps no smoothing, so the submitted pipeline is unsmoothed.
2 vs 3 TCN seeds (CPU limit)	Runtime only: one seed ≈ 40 s on the GPU run (44, 40, 30 s for seeds 0, 1, 2); AUC delta not measured	Seed averaging is cheap insurance at the observed per-seed cost; the hold-out AUC gain was not measured separately.

Notebook 2 Appendix B: Complete feature dictionary (86 variables)

The names follow the feature construction in `Short_Notebook_2.ipynb`. Group sizes: price shape 22, calendar 7, per-unit economics 22, scale-free margin 7, reservoir group state 6, recent history 3, system water 2, unit data 5, hydro budget 5, hourly environmental limits 7.

#	Feature(s)	Meaning
1–5	`p_std`, `p_q90`, `p_q10`, `p_spread`, `p_peak_hour`	Weekly dispersion, inter-quantile spread and location of the weekly price peak.
6–13	`price`, `p_rank`, `p_minus_mean`, `p_z`, `p_c6`, `p_c24`, `p_accel`, `p_vol6`	Hourly price and its position, smoothing and variability within the week.
14	`p_day_rank`	Rank of the hour's price within its own day.
15–22	`p_lag1/3/12/24`, `p_lead1/3/12/24`	Price 1, 3, 12, 24 h before / after (tree-visible neighbour context).
23–29	`hour_of_day`, `weekday`, `is_weekend`, `doy_sin`, `doy_cos`, `t`, `unit`	Real calendar features; week position and unit index.
30–39	`u_up_W7`, `u_dn_W7`, `u_up_W0`, `u_dn_W0`, `u_up_W14`, `u_dn_W14`, `u_up_V0`, `u_dn_V0`, `u_up_Iwk`, `u_dn_Iwk`	Aggregated upstream/downstream water values (d, $d+7$, $d+14$), volumes and weekly inflows.
40–41	`u_netW7`, `u_netW14`	Upstream minus downstream terminal water value.
42	`m`	Core margin: price $-$ WV($d{+}7$) of the upstream reservoir.
43–49	`m_cmean7`, `m_cmean25`, `m_frac_week`, `m_dprev`, `m_dnext`, `m_wq20`, `m_wq80`, `m_pos_sum`	Temporal context of the margin: centred means, positive fraction, distances to sign changes, weekly quantiles, positive sum.

#	Feature(s)	Meaning
50	`m14`	Price $-$ WV($d+14$) of the upstream reservoir.
51–55	`m_rank`, `m_z`, `m_x_prank`, `m_over_pstd`, `u_ratio`	Scale-free margin variants and the price-to-water-value ratio.
56–57	`m_units_mean`, `m_minus_units`	Cross-unit margin context at the same hour.
58–63	`g_fill`, `g_inflow_over_cap`, `g_spill_risk`, `g_headroom_days`, `g_inflow_h`, `g_inflow_c24`	Effective storage of the tunnel-connected group: fill, inflow ratios, spill risk, headroom, hourly and 24 h-centred inflow.
64–66	`u_dV7`, `u_inflow_prev7`, `u_inflow_prev30`	7-day filling trend; mean inflow of the 7 / 30 days before the case.
67–68	`sys_total_inflow`, `sys_total_fill`	Catchment-wide inflow and stored volume.
69–73	`gen_pmax`, `gen_start_per_mw`, `plant_n_units`, `unit_idx_in_plant`, `m_minus_startcost`	Unit statics and margin net of normalised start cost.
74–78	`hb_storage_local_h`, `hb_inflow_local_h`, `hb_headroom_local_h`, `hb_spill_local_h`, `hb_inflow_unit_h`	Hydro budget in hours of full plant / unit discharge.
79–85	`mfh_b_Byrtevatn`, `mfh_r_Vest_Vassdraget`, `rvest_deficit_h`, `rvest_deficit_over_lio`, `mvmargin_Totak`, `mvmargin_Byrtevatn`, `mvh_Byrtevatn`	Hourly minimum flows, the Lio receiving-river deficit, minimum-volume margins.